

Sustainable Kubernetes Extension Stack

Final Year Project

George Lipceanu (20103125)

May 5th, 2026



Abstract

This *Sustainable Kubernetes Extension Stack (SusK8s)* is a set of Kubernetes-native extensions that allow Kubernetes to treat energy and carbon metrics as scheduling and reporting inputs. The stack introduces Custom Resource Definitions to represent real-time carbon intensity (**CarbonInfo**), policy-driven workload preferences (**WorkloadPolicy**), and auditable reports through (**EmissionReport**). The declarative state of these resources is maintained by a central **SusK8s Operator**, which manages the system through **five specialised controllers**: a Policy Controller for workload annotation, a Carbon Sync Controller for real-time grid data ingestion, a Metrics Controller for power and carbon aggregation, a Rescheduling Controller for reactive and proactive pod eviction, and a Reporting Controller for audit generation. To enforce policy compliance, a Mutating Admission Webhook intercepts workloads at the API level, dynamically injecting sustainability boundaries and scheduling constraints before pods are persisted.

Sustainability intent is enacted at placement time via **susk8s-scheduler**, an independent custom scheduler built upon the Kubernetes Scheduling Framework. Operating as a distinct alternative to the default scheduler, this dedicated instance utilises a **CarbonFilter** plugin to respect hard emission limits, and a **CarbonBinPack** scoring plugin to favour environmentally optimal nodes. For measurement, Kepler estimates container energy usage via eBPF, with Prometheus, Kube-Ops-View, and Grafana providing a real-time observability pipeline.

To make the stack easier to use, a dedicated Command Line Interface (CLI) developed using the Go Cobra CLI framework provides an alternative interface for managing the stack. The **susk8s** CLI wraps installation, resource creation, reporting, and port-forwarding commands, allowing users to **install** the stack, **create** and **delete** resources, query live **carbon** metrics, extract **report** audits, and access the observability ui. The system is validated using an A/B evaluation to compare the custom scheduler to default Kubernetes scheduling algorithms and how the stack can be improved in future work.

Acknowledgements

I would like to give a sincere thanks to my supervisor, Richard Frisby, whose support and insights were vital to this project. His mentorship has been a cornerstone of my learning this year, and I am incredibly appreciative of his assistance over the semester.

I would also like to give a sincere thanks to Lucy White for assisting me through the submission process and to the my panel, Richard Lacy, Bernard Butler and Opeyemi Bamigbade for taking the time to review my work and be on the panel for my project presentation.

Contents

1	Introduction	11
1.1	Motivation and Purpose	13
1.2	Research Questions	14
2	Literature Review	14
2.1	Energy and Power Modelling	14
2.2	Telemetry and Measurement	15
2.3	Sustainable Scheduling and Policy	15
2.4	Green Software Standards and Carbon Metrics	16
2.5	The Operator Pattern and Extensible Control Planes	16
2.6	Admission Control and Proactive Governance	16
2.7	UN Sustainable Development Goals	16
3	Methodology	18
3.1	Agile	18
3.2	Sprint Planning and Task Tracking	18
3.3	Version Control	18
3.4	Supervisor Meetings	18
3.5	User Stories	18
3.5.1	User Story 1	18
3.5.2	User Story 2	19
3.5.3	User Story 3	19
4	System Architecture and Design	20
5	Individual Components	23
5.1	CRDs	23
5.2	Operator Controller Logic	28
5.3	Scheduler Design and Implementation	31
5.3.1	Implementation Structure	31
5.3.2	Configuration and Extensibility	31
5.3.3	Modified Scheduler Design for Sustainability	32
5.3.4	Evolution Scheduler Architecture	33
5.3.5	Scheduler Plugin Implementation and Logic	34
5.4	Additional Tooling	35
5.4.1	Kepler Integration as the Measurement Layer	35
5.4.2	Prometheus	35
5.4.3	Grafana	35
5.4.4	Kube-Ops-View	39
5.4.5	Cobra CLI	39
6	Testing and Evaluation	40
6.1	Scheduler Unit Testing	40
6.2	Operator and Webhook Unit and Integration Tests	40
6.3	End-to-End Evaluation	41
6.4	SusK8s vs Vanilla Kubernetes A/B Evaluation	42
7	System Deployment and User Guide	44
7.1	Prerequisites	44
7.2	Deploying the Observability and Telemetry Stack	44
7.3	Deploying the SusK8s Operator and CRDs	45
7.4	Deploying the Custom Scheduler	45
7.5	Building and Using the CLI	46
7.6	Video Demo	46

8	Future Work	47
8.1	Carbon-Aware Autoscaler	47
8.2	Multi-Region Cluster Routing	47
8.3	Additional Observability Options	48
8.4	Improved RBAC Roles and Security in Cluster Environment	49
8.5	Other Kubernetes Deployments	49
8.6	Directory and File Refactoring	49
9	Summary and Discussion	50

List of Figures

1	High level overview of how Prometheus and Grafana interact with each other in a Kubernetes environment [1].	15
2	The Kubernetes operator controller reconciliation loop.[8].	16
3	The UN Sustainable Development goals.[27].	17
4	Initial architecture of the Sustainable Kubernetes Extension Stack.	20
5	Current architecture of the Sustainable Kubernetes Extension Stack. (NOTE: The Mutating Admission Webhook runs inside of the custom operator but is detached from it here for clarity.	22
6	<code>susk8s-operator</code> diagram	30
7	Kubernetes Scheduler flow, with <code>CarbonBinPack</code> scoring logic included	33
8	<code>susk8s-scheduler</code> flow.	34
9	Custom SusK8s Grafana dashboards.	37
10	Kepler Grafana dashboards displaying no metrics 1.	38
11	Kepler Grafana dashboards displaying no metrics 2.	38
12	Kepler Grafana dashboards displaying no metrics 3.	39
13	Kube-Ops-View visualisation from the browser.	39
14	Unit tests passing on scheduling plugins with over 90% coverage.	40
15	Unit tests passing on operator controllers with over 70% coverage. (NOTE: The files in <code>internal/controller</code> aren't actually used and are residual files which were not removed due to concerns with potentially breaking something or needing those files once again).	41
16	E2E tests passing	42
17	Grafana time-series graph of current emissions being produced by the worker nodes during Vanilla Kubernetes evaluation.	42
18	Cluster Carbon Emissions: Vanilla kube-scheduler vs. SusK8s Operator during the 15-minute Brownout experiment.	43
19	Karmada system architecture diagram, showing how the global control plane interacts with the multi-cluster deployment [9].	47
20	Thanos diagram with Prometheus and Grafana integration, showing that a deployment that incorporates both long and short-term time stores is feasible [29].	48

SETU Anti-Plagiarism Declaration

I certify that this assignment is all my own work and contains no plagiarism. By submitting this assignment, I agree to the following terms: Any text, diagrams or other material copied from other sources (including, but not limited to, books, journals and the internet) have been clearly acknowledged and referenced as such in the text by the use of ‘quotation marks’ (or indented italics for longer quotations) followed by the author’s name and date [eg (Byrne, 2018)] either in the text or in a footnote/endnote. These details are then confirmed by a fuller reference in the bibliography. I have read Appendix 1 of the SETU Student Academic Misconduct Policy and Disciplinary Procedure and I understand that only assignments which are free of plagiarism will be awarded marks. I further understand that the disciplinary procedure can lead to the suspension or permanent expulsion of students in serious cases.

GenAI Statement

Generative artificial intelligence (GenAI) was used in this work. I, the author, have reviewed and edited all content and take full responsibility for the content, claims, and references presented in this report and in the project’s codebase. An overview of GenAI use is provided below.

Tool	Purpose	Extent of Use
ChatGPT	Quick debugging, architectural questions and relevant sources for code and infrastructure. Quick layout changes to the report	Used before the first demo with supervisor in week 4 due to debug issue with breaking ServiceMonitors. Also used to consult alternatives to reconciling controllers when it came to assigning <code>WorkloadPolicy</code> annotations to Pods. Also queried on sources to investigate regarding Mutating Admission Webhooks and the Kubernetes Scheduling in general. Also used to sort the Glossary of this report alphabetically.

Signed: George Lipceanu

Date: 2026-05-06

Glossary

Table 1: Glossary of core architectural components and concepts.

Term	Definition
A/B Testing	A comparative experimental methodology where two versions of a service or configuration are run simultaneously to determine which performs better.
AdmissionReview	The Kubernetes API object used during admission control to pass resource creation requests to admission webhooks for validation or mutation.
API Server	The central Kubernetes control-plane component that exposes the Kubernetes API and acts as the primary interface for cluster communication and management operations.
Bin-Packing	A workload placement strategy that consolidates workloads onto fewer nodes to maximise utilisation and reduce wasted infrastructure capacity.
Carbon Intensity	The quantity of greenhouse gas emissions produced per unit of electricity generated, typically expressed in grams of CO_2 equivalent per kilowatt-hour (gCO_2e/kWh).
CarbonBinPack	A custom SusK8s scheduler scoring plugin that combines node utilisation and carbon intensity metrics to prioritise environmentally efficient workload placement.
CarbonFilter	A custom SusK8s scheduler filter plugin that evaluates node eligibility against workload carbon-intensity constraints before scheduling decisions are made.
Cobra CLI	A library for creating powerful modern CLI applications in Go, used to provide a structured command-line interface that abstracts complex backend workflows into simplified user commands.
Control Plane	The collection of Kubernetes components responsible for maintaining cluster state, scheduling workloads, and enforcing orchestration logic.
CRD (Custom Resource Definition)	Kubernetes-native APIs used within this stack to capture sustainability signals and preferences, complete with OpenAPI validation, defaults, and standard status conditions.
DaemonSet	A Kubernetes workload controller that ensures a specific Pod runs on all (or a selected subset of) Nodes within a cluster, commonly used for deploying background services such as log collectors, monitoring agents, or storage drivers.
Day 0 (Design & Architect)	The initial phase focused on defining requirements, selecting infrastructure, and architecting the sustainability boundaries before any code is deployed.
Day 1 (Deploy & Configure)	The implementation phase involving the initial installation, provisioning, and configuration of the software stack and its integration into the target environment.
Day 2 (Maintain & Optimise)	The long-term operational phase focused on continuous monitoring, maintenance, scaling, and the optimisation of systems based on real-world performance data.

Term	Definition (Continued)
Docker	An open-source platform that uses OS-level virtualization to deliver software in packages called containers, providing the tooling to build, deploy, and manage containerized applications efficiently.
E2E (End-to-End) Testing	A comprehensive validation methodology that tests a system’s entire workflow from start to finish, ensuring that all integrated components and external dependencies function correctly in a production-like environment.
eBPF (<i>Extended Berkeley Packet Filter</i>)	A technology that allows programs to run in a protected environment inside the operating system kernel, enabling high-performance monitoring, networking, and security observability without modifying kernel source code or loading traditional modules.
etcd	The distributed key-value store used by Kubernetes as its primary backing datastore for all cluster state and configuration data.
Eviction API	A Kubernetes programmatic interface that allows for the graceful termination of Pods while respecting Pod Disruption Budgets (PDBs), ensuring that workload availability is maintained during voluntary disruptions.
<i>gCO₂e</i>	Grams of carbon dioxide equivalent, a standardised unit used to measure greenhouse gas emissions across different gases.
Go (Golang)	An open-source, statically typed programming language designed for simplicity and efficiency, widely used for cloud-native development due to its robust concurrency primitives and compiled performance.
Grafana	An open-source multi-platform data visualisation and monitoring tool used to render operational data and metrics into interactive, real-time dashboards.
High Availability (HA)	A system design approach focused on minimising downtime and ensuring continuous service operation during failures or maintenance events.
Idempotency	A property of certain operations whereby they can be applied multiple times without changing the result beyond the initial application; in the context of Kubernetes, it ensures that reconciliation loops can safely repeat actions to reach a desired state.
Informer / SharedInformer	A component of the Kubernetes client library that maintains a local, in-memory cache of cluster resources and reacts to changes via events, providing high-performance, low-latency data access without repeatedly querying the API server.
Kepler (<i>Kubernetes Efficient Power Level Exporter</i>)	A specialised metrics exporter that utilises eBPF-based kernel instrumentation and hardware performance counters to estimate and report energy consumption at the process and container level.
KinD (Kubernetes in Docker)	A tool for running local Kubernetes clusters using Docker container nodes, primarily designed for testing Kubernetes itself or local development environments.
Kube-ops-view	A read-only system-wide visualisation tool that provides a high-level overview of a Kubernetes cluster, displaying node capacity, pod scheduling, and resource saturation in a grid-based dashboard.
Kubebuilder	An SDK for building Kubernetes APIs using the Operator Pattern, providing a framework to quickly develop and test Custom Resource Definitions (CRDs) and controllers.

Term	Definition (Continued)
Kubernetes	An open-source container orchestration platform designed to automate the deployment, scaling, and management of containerised applications across a cluster of hosts.
Kubernetes Scheduling Framework	A pluggable architecture within the default Kubernetes scheduler that allows developers to extend scheduling capabilities by implementing custom logic at various stages of the scheduling cycle.
Mutating Admission Webhook	A Kubernetes controller that intercepts requests to the API server before an object is persisted, allowing for the dynamic modification or "mutation" of the resource's configuration to enforce defaults, security standards, or operational boundaries.
Namespace Selector	A label-based query mechanism used to identify and filter a subset of namespaces for the application of cluster-wide resources, such as NetworkPolicies, RBAC roles, or admission controller configurations.
Operator Pattern	A Kubernetes software design pattern that encapsulates human operational knowledge into software to automate the management of complex, stateful applications by reconciling actual state with desired state.
Pending State	A Kubernetes pod lifecycle phase indicating that the pod has been accepted by the cluster but has not yet been scheduled or started.
Pod	A deployable unit in Kubernetes, representing a single instance of a running process in a cluster. It can contain one or more containers that share storage, network resources, and a specification for how to run.
PodDisruptionBudget (PDB)	A Kubernetes API object used to ensure high availability by limiting the number of concurrent voluntary disruptions to a set of replicated pods during maintenance or administrative tasks.
Policy-as-Code (PaC)	A governance approach where operational and security policies are expressed declaratively as machine-readable configuration rules.
Prometheus	An open-source monitoring and alerting toolkit designed for reliability and scalability, functioning as a time-series database and metrics collection engine.
PromQL	A functional query language utilized by the Prometheus monitoring system to select, aggregate, and analyse time-series data in real-time.
Reconciliation Loop	The continuous control cycle used by Kubernetes controllers and operators to compare the current cluster state with the desired declarative state and take corrective actions when drift occurs.
Rolling Restart	A deployment update strategy where application instances are restarted incrementally to minimise service disruption.
Scope 1 Emissions	Direct greenhouse gas emissions from sources that are owned or controlled by an organisation, such as emissions from onsite fuel combustion or company-owned vehicle fleets.
Scope 2 Emissions	Indirect greenhouse gas emissions associated with the purchase of electricity, steam, heat, or cooling consumed by the organisation's operations.
Scope 3 Emissions	All other indirect emissions that occur in an organisation's value chain, including both upstream and downstream activities such as purchased goods, services, business travel, and product end-of-life.

Term	Definition (Continued)
Time-Series Database	A database optimised for storing and querying timestamped metrics data, such as Prometheus monitoring metrics.
Unit Testing	A software testing method where individual components or functions of an application are isolated and tested independently to verify that each part performs correctly according to its specifications.
Watch API	A Kubernetes API mechanism that streams real-time resource changes to clients such as controllers and schedulers.

1 Introduction

This project delivers a sustainability extension stack for Kubernetes. It integrates *Custom Resource Definitions (CRDs)*, *reconciling controllers*, and *carbon-aware scheduler plugins* into a cohesive system that makes energy, carbon emissions and sustainability first-class signals in cluster operations. A companion CLI automates installation, upgrades, and cluster validation, while an observability layer (Prometheus, Grafana and Kube-Ops-View) exposes real-time power, energy, and emission metrics.

Problem Statement Kubernetes optimises for resource fit, availability, and fairness, but lacks native constructs to reason about energy or carbon. Operators cannot declaratively express sustainability intent, like favouring lower-intensity nodes, or audit the energy and emissions impact of workloads without ad-hoc tooling. This gap creates three distinct challenges for modern infrastructure:

- **The Observability Gap:** Kubernetes does not natively correlate container resource consumption (CPU/RAM) with physical energy draw (Joules) or real-time grid telemetry (gCO_2/kWh), preventing granular Scope 3 emissions auditing.
- **The Placement Gap:** The default scheduling framework evaluates nodes based on available capacity rather than environmental efficiency, leading to carbon-agnostic bin-packing that ignores cleaner geographical regions.
- **The Reconciliation Gap:** Grid carbon intensity is highly volatile, yet default Kubernetes scheduling is a point-in-time decision. There is no native mechanism to proactively evict and reschedule workloads when a local grid experiences a sudden spike in carbon intensity.

Goals The main goal of the Sustainable Kubernetes Extension Stack (SusK8s) is to make sustainability defined through APIs, enforced by the scheduler, and measured through report for modern infrastructure. To achieve this, the architecture shifts carbon-awareness from external, ad-hoc scripts to the core Kubernetes control plane by focusing on three foundational pillars:

1. **Modelling Carbon and Energy as Native API Resources:** To integrate seamlessly with existing cloud-native workflows, environmental telemetry must be represented as Kubernetes API resources. By introducing Custom Resource Definitions (CRDs) such as `CarbonInfo`, `WorkloadPolicy`, and `EmissionReport`, the system translates highly volatile, external grid metrics into declarative, etcd-backed objects. This enables infrastructure operators to define, query, and enforce energy constraints using the exact same standard API constructs and `kubectl` tooling they use to manage CPU and memory.
2. **Reconciling Cluster State Toward Sustainability Intent:** The core philosophy of Kubernetes relies on the continuous, autonomous reconciliation of cluster state. The SusK8s Operator extends this paradigm by deploying five specialised controllers that constantly evaluate an operator's desired sustainability state (defined by workload policies) and the current cluster state. When local grid carbon intensity exceeds permissible thresholds, the Rescheduling Controller reacts dynamically, enforcing strict policy compliance by systematically evicting pods to migrate workloads to cleaner nodes without requiring manual human intervention.
3. **Driving Intelligent Scheduling and Auditable Reporting:** Sustainability signals are only valuable if they drive both proactive placement and empirical accountability. At the deployment phase, a Mutating Admission Webhook and a dedicated custom scheduler (`susk8s-scheduler`) proactively route workloads to environmentally optimal nodes based on `CarbonFilter` limits and `CarbonBinPack` scoring. Post-deployment, the architecture utilises eBPF-based measurements via Kepler to track real-time energy draw. These granular signals are subsequently aggregated into formal, auditable `EmissionReport` receipts and visualised via a Grafana observability pipeline, enabling strict Scope 3 emissions tracking.

Scope The SusK8s stack provides:

- **APIs and Admission Control:** A suite of Custom Resource Definitions (`CarbonInfo`, `WorkloadPolicy`, and `EmissionReport`) for sustainability management, supplemented by a **Mutating Admission Webhook** that ensures real-time policy and annotation injection at the API level.
- **SusK8s Operator:** A central management engine orchestrating **five specialised controllers**:
 - **Workload Policy Controller:** Acts as a passive observer that discovers targeted deployments and dynamically updates the policy status (e.g., `matchedWorkloads`), leaving state mutation to the admission webhook to ensure idempotency and prevent disruptive rolling restarts.
 - **Carbon Info Controller:** Ingests live grid intensity data from external providers.
 - **Kepler Metrics Controller:** Aggregates Kepler eBPF-based energy signals into node-level telemetry.
 - **Rescheduling Controller:** Performs reactive and proactive, eviction-based corrective actions or “rescheduling”.
 - **Reporting Controller:** Generates auditable sustainability summaries and savings data.
- **Custom Scheduler Instance:** An independent `susk8s-scheduler` built on the Kubernetes Scheduling Framework. It features two plugins, a filter plugin called `CarbonFilter` for hard-limit enforcement on pods violating workload policies and a scoring plugin called `CarbonBinPack` regional carbon intensity scheduling weights (which the user configures) for environmentally optimal pod-to-node bindings.
- **Observability and Telemetry Stack:** Integration with Kepler for eBPF-driven power estimation, Prometheus for time-series metric collection, a custom Grafana dashboard for real-time energy and sustainability monitoring, Kube-Ops-View for a live visualisation of pod placements on every node in the cluster.
- **Unified Management Tooling:** A dedicated `susk8s` Cobra-based CLI providing a single entry point to `install` the stack for day 2 cluster installation (and `create` a cluster with the stack installed from Day 0), manage `carbon` data, generate `report` audits, and set up all the port forwarding for the observability `ui` with a single command.
- **Testing and Evaluation Framework:** A comprehensive testing suite ensuring system reliability. Unit testing is applied across all five controllers, the mutating webhook, and the custom scheduler plugins. It is supported by E2E test scenarios that verify the full lifecycle of policy injection, placement, and eviction. An A/B “Vanilla vs. SusK8s” testing environment to test carbon reduction in a SusK8s cluster against a default Kubernetes baseline cluster.

Design Principles The architecture of the SusK8s stack is guided by six core systems engineering principles:

1. **Upstream Compatibility:** The system is built entirely upon official Kubernetes extension mechanisms, specifically CRDs, webhooks, and the Scheduling Framework, ensuring that no modifications to the core Kubernetes source code are required.
2. **Separation of Concerns:** The system’s architecture strictly isolates operational duties to prevent cascading failures. The SusK8s Operator decouples core control logic into five controllers (handling telemetry, policies, rescheduling, live regional carbon data, and reporting), while initial workload placement and user administration are handled by entirely independent components (the `susk8s-scheduler` and the CLI, respectively).
3. **Declarative Intent:** Sustainability constraints, scheduling weights, and carbon limits are defined strictly through CRDs (e.g., `WorkloadPolicy`), acting as the single source of truth for cluster environmental goals.

4. **Policy Enforcement:** Sustainability boundaries are enforced proactively at the API level via a Mutating Admission Webhook, and maintained by the SusK8s Operator’s **Rescheduling Controller** as sustainability metrics such as grid conditions fluctuate.
5. **Observability by Default:** Environmental impact is never a black box. All data from eBPF-derived container power metrics via Kepler to cluster-wide Scope 3 emissions are continuously exposed through Prometheus, Grafana, Kube-Ops-View and structured **EmissionReport** audits.
6. **Fault Tolerance:** The custom scheduler consumes pre-computed sustainability metrics which are produced by the SusK8s Operator. These metrics are consumed directly from the local API cache to guarantee low-latency placement decisions. Pod evictions strictly respect native safety constraints, such as **PodDisruptionBudgets**, ensuring sustainability does not compromise high availability.

Implementation Overview The system is implemented entirely in Go, using Kubebuilder to scaffold the SusK8s Operator and its five controllers and webhook, alongside the Kubernetes Scheduling Framework to construct the custom scheduler instance. The workload lifecycle begins at the API level, where a Mutating Admission Webhook intercepts deployment requests to dynamically inject declarative sustainability constraints. During the scheduling phase, the **susk8s-scheduler** employs the previously mentioned (**CarbonFilter** for hard-limit enforcement and **CarbonBinPack** for optimal scoring in relation to the metrics provided by the Workload Policy. As mentioned before, the scheduler consumes pre-computed node sustainability labels projected into the local API cache by the control plane, and does not query external APIs to maintain low-latency placement. This low-latency data projection is achieved via the Kubernetes Informer pattern. Supporting controllers patch sustainability metrics directly onto **Node** objects as labels or annotations. The custom scheduler, maintaining a persistent Watch connection to the API server, automatically syncs these updates into its local, in-memory **SharedInformer** cache, ensuring that scoring plugins execute in microseconds without blocking on network I/O.

Post-placement, the Operator’s Rescheduling Controller continuously monitors live node conditions against WorkloadPolicy thresholds, enforcing long-term compliance via controlled pod evictions. The system’s observability pipeline utilises Kepler for eBPF-derived power estimation, aggregating time-series metrics via Prometheus for real-time visualisation in Grafana and Kube-Ops-View. Finally, a Cobra-based CLI provides users with a single entry point for Day 0/Day 2 cluster provisioning, simple carbon emission calls for different regions, telemetry port-forwarding, and the generation of auditable **EmissionReport** receipts.

1.1 Motivation and Purpose

Cloud workloads in Kubernetes are energy-intensive but the platform obscures its carbon impact. Teams cannot easily see anything regarding sustainability in their workloads, such as where workloads ran or how much energy they consumed. AI/ML and data-ingesting and processing workloads are growing rapidly in cloud environments, making a practical way to measure, optimise, and report sustainability at the orchestration layer timely and needed in order to maintain sustainability. This project addresses that gap through three core capabilities:

1. **Measurement:** eBPF-driven power estimation via Kepler, aggregating precise, container-level energy consumption into node telemetry.
2. **Enforcement:** Layered guardrails consisting of a Mutating Admission Webhook for proactive API-level rejection, and a Rescheduling Controller for runtime pod evictions.
3. **Optimisation:** A custom scheduler instance that actively reduces emissions by combining utilisation-aware bin-packing with real-time carbon intensity scoring.
4. **Reporting & Observability:** CRD-driven **EmissionReport** audits, real-time Grafana dashboards, live Kube-Ops-View pod visualisations and unified CLI tooling to bridge the gap between compliance and platform engineering.

The specific focus on the Kubernetes platform came as a result of my 8 month internship at Red Hat last year from 2025 where I was put on the OpenShift control planes team. On this team, I learned about technologies used for this project, such as Kubernetes, Docker and containers in general, Golang, creating, modifying, testing and deploying Kubernetes operators using Kubebuilder, and Cobra CLI. Because of the previously mentioned gap in Kubernetes regarding sustainability observation auditing and my experience in some relevant technologies, I felt positioned to develop a solution that integrates sustainability directly into the cluster lifecycle, moving beyond simple observation toward automated, policy-driven carbon management.

1.2 Research Questions

To evaluate the effectiveness of the Sustainable Kubernetes Extension Stack (SusK8s) in addressing the observability, placement, and reconciliation gaps, this research is guided by the following primary questions:

1. **Scheduling Optimisation:** How much can we reduce the carbon emissions of cloud workloads by making the core system scheduler actively aware of real-time grid conditions compared to the default Kubernetes scheduler?
2. **Dynamic Reconciliation:** How can a system safely and automatically relocate running applications when local grid emissions spike, without causing unacceptable downtime for users?
3. **Architectural Decoupling:** What are the system performance benefits of measuring energy usage after an application is placed, rather than trying to calculate it during the fast-paced scheduling process?

2 Literature Review

The literature surrounding this project focuses on integrating environmental signals into cloud-native orchestration platforms. Building upon the foundational implementation detailed in the initial project report, which established the core telemetry pipeline and a custom sustainability aware scheduler, this architecture has evolved into a comprehensive sustainable control plane. As already mentioned, Kubernetes natively optimises for resource availability, throughput, and fairness[13], but it lacks mechanisms for expressing or enforcing sustainability intent such as energy efficiency or carbon awareness. To address this gap, the following review gathers the concepts required to support the entire end-to-end scope.

2.1 Energy and Power Modelling

Accurate power estimation is a cornerstone of sustainability-aware scheduling and reporting. This project adopts a hardware-aware linear CPU power model to estimate node-level power consumption. The instantaneous power draw of a node E_n is calculated as:

$$E_n = P_{\text{idle}} + U_{\text{cpu}} (P_{\text{max}} - P_{\text{idle}}) \quad (1)$$

where P_{idle} represents the idle power of the node, P_{max} its maximum power draw, and U_{cpu} the current CPU utilisation ratio. This linear model, supported by prior research from Lin et al. and Rivoire et al., offers a practical balance between computational simplicity and sufficient accuracy for cloud-scale sustainability analysis [17, 23].

However, relying on static assumptions for P_{idle} and P_{max} introduces inaccuracies in highly changing cloud environments. Because of this, this project has now transitioned away from this theoretical mathematical models in favour of direct, empirical hardware telemetry. As will be explored in greater detail in the following sections, the system bypasses the linear power model entirely by using Kepler to capture energy consumption data. This approach eliminates the need for estimates by querying the exact hardware energy consumption directly from the kernel, which greatly increases the precision and reliability of the platform’s `EmissionReport` audits and Grafana dashboards.

2.2 Telemetry and Measurement

Modern sustainability observability relies on lightweight, non-intrusive measurement techniques. Kepler utilises eBPF-based kernel instrumentation and hardware performance counters to estimate node- and container-level energy consumption [24]. This approach avoids invasive monitoring while providing granular energy metrics suitable for large-scale deployments. Kepler metrics are typically exported to Prometheus, enabling time-series analysis and aggregation for higher-level sustainability reporting [22].

To bridge the gap between raw telemetry and actionable operational intelligence, modern orchestration platforms rely on advanced presentation layers. Grafana integrates natively with Prometheus to render dynamic, customisable dashboards, translating aggregated energy data into visual sustainability trends and compliance metrics [4].

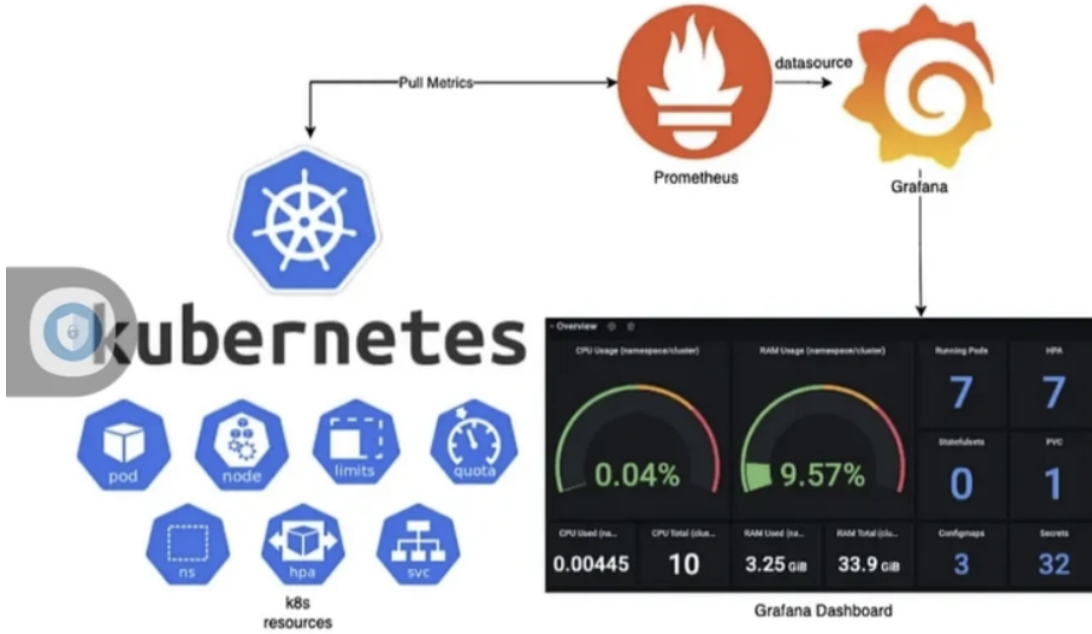


Figure 1: High level overview of how Prometheus and Grafana interact with each other in a Kubernetes environment [1].

Topological visualisation tools such as `kube-ops-view` provide real-time, spatial mapping of cluster state, including node capacity utilisation and live pod distribution [7]. In the context of sustainability-aware orchestration, these topological interfaces are vital for observing the immediate effects of utilisation-aware bin-packing and pod evictions across nodes with varying carbon intensities.

2.3 Sustainable Scheduling and Policy

Prior work on sustainable scheduling proposes extending placement logic with carbon intensity signals obtained from external providers such as ElectricityMap [3]. By leveraging the Kubernetes Scheduling Framework, custom scheduling plugins can be implemented without modifying upstream components. Plugins such as the custom *CarbonBinPack* created for this project combine utilisation-aware bin-packing with carbon-aware scoring to prioritise well-utilised, low-carbon nodes while maintaining compatibility with standard Kubernetes scheduling behaviour [21, 28].

In addition, the use of CRDs enables sustainability constraints and policies to be expressed declaratively, shifting governance from ad-hoc scripts toward auditable, API-driven mechanisms [11]. This modular, Kubernetes-native approach allows sustainability-aware rescheduling to occur after initial

placement, ensuring that workloads remain compliant as environmental conditions change, without compromising reliability or availability guarantees [12].

2.4 Green Software Standards and Carbon Metrics

The shift towards sustainable cloud computing is a result of the work done by organisations such as the Green Software Foundation. A critical metric in this domain is Grid Carbon Intensity (typically measured in gCO_2/kWh), which quantifies the greenhouse gas emissions associated with the electricity consumed at a specific time and location [6]. Literature distinguishes between average and marginal carbon intensity, the latter being particularly relevant for orchestration platforms as it reflects the emissions of the power plants that would be spun up or down to meet scaling cloud workloads [18]. By tracking grid intensity, scheduling algorithms can perform spatial and temporal (delaying workloads until the grid is greener) carbon optimisation.

2.5 The Operator Pattern and Extensible Control Planes

Extending Kubernetes to handle custom sustainability logic requires robust systems architecture. The Operator pattern, introduced by CoreOS, has become the industry standard for encoding domain-specific operational knowledge into Kubernetes [14]. Operators are built upon the Kubernetes control theory model, utilising continuous reconciliation loops to observe the cluster’s current state, compare it against a desired state (defined via CRDs), and take actions to eliminate any drift [8].

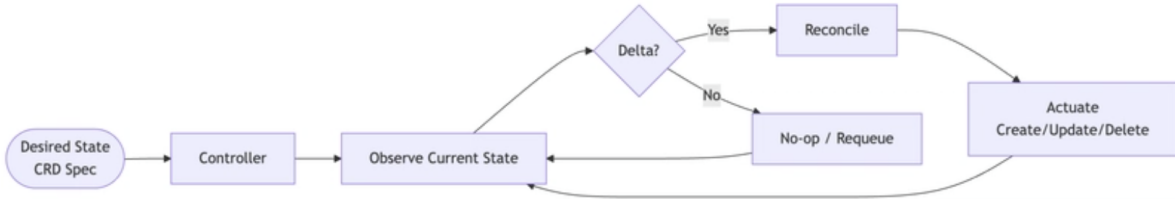


Figure 2: The Kubernetes operator controller reconciliation loop.[8].

Decoupling control logic into isolated, specialised controllers ensures fault tolerance and prevents cascading failures, which is critical when ingesting volatile external telemetry such as live power metrics or grid carbon data.

2.6 Admission Control and Proactive Governance

While custom schedulers handle workload placement, modern Kubernetes governance relies heavily on Admission Control to enforce organisational policies before objects are persisted to `etcd` [19]. Literature on Policy-as-Code (PaC) demonstrates that mechanisms such as Mutating and Validating Admission Webhooks serve as crucial proactive guardrails [25]. By intercepting API requests, webhooks can dynamically mutate workloads to inject required sustainability metadata, such as scheduling tolerances or resource limits, ensuring that no workload enters the cluster without adhering to the declared environmental baseline. This layered enforcement approach shifts sustainability left, embedding it directly into the deployment lifecycle.

2.7 UN Sustainable Development Goals

The 2030 Agenda for Sustainable Development provides a shared blueprint for peace and prosperity for people and the planet [27]. At its core are the 17 Sustainable Development Goals, which represent an urgent call for action by all countries in a global partnership.



Figure 3: The UN Sustainable Development goals.[27].

As the global reliance on digital infrastructure accelerates, the computing industry, particularly cloud computing and data centers, accounts for a rapidly growing percentage of global electricity demand. As a result, modern software engineering and platform architecture must actively align with these global frameworks. The development of the SusK8s carbon-aware orchestration platform directly addresses and advances three critical SDGs:

- **Goal 9: Industry, Innovation, and Infrastructure.** A core target of SDG 9 is to upgrade infrastructure and fit these to industries to make them sustainable, with increased resource efficiency and greater adoption of clean, sustainable technologies. By injecting real-time carbon telemetry into Kubernetes, this project innovates upon standard cloud infrastructure, turning static clusters into dynamic, environmentally responsive systems.
- **Goal 12: Responsible Consumption and Production.** SDG 12 is centered on “doing more and better with less,” urging the decoupling of economic growth from environmental degradation. In this project, this is implemented through the CarbonBinPack scheduling plugin and the Rescheduling Controller, which together aim to reduce unnecessary node usage and move workloads away from higher-carbon nodes. By enforcing strict bin-packing algorithms to maximise node utilisation and actively evicting workloads during periods of high grid carbon intensity, the system prevents the wasteful over-provisioning of cloud compute resources.
- **Goal 13: Climate Action.** SDG 13 demands urgent action to combat climate change and its impacts. While cloud providers often rely on carbon offsets to claim “net-zero” operations, this project focuses on genuine, active carbon mitigation. Through the **EmissionReport** CRD and hardware-level eBPF metrics (Kepler), the platform provides transparent, auditable spatial attribution of greenhouse gas emissions (gCO₂), encouraging organisations to tangibly measure and reduce their direct software footprint.

3 Methodology

This section tackles the project management methodologies and engineering workflows used throughout the development lifecycle. It outlines how the project was approached on a week-to-week basis regarding iterative development, version control, feedback loops with the project supervisor, and requirement engineering.

3.1 Agile

The project adopted an Agile software development methodology where development was broken down into iterative week long sprints. This approach was critical for managing the project's technical risks, as it allowed for continuous integration and rapid pivoting when architectural roadblocks were encountered, such as the realisation that controllers can't inject pods with annotations before they are scheduled.

3.2 Sprint Planning and Task Tracking

Trello was used as the primary project management tool to track engineering workflows and manage sprint cycles. Initially Jira was the primary project management tool however after countless errors when it came to ticket creation in this project board specifically, project management was migrated to Trello for its simplicity and ease of use. The project scope was broken down into actionable tickets and organised into logical stages, typically reflecting *Current Sprint/In Progress*, *On Hold*, *Next Up*, and *Done*. This ticket-driven approach provided benefits throughout the project lifecycle by helping with scope management, prioritisation and progress visualisation as tickets are created, completed and moved from one stage to the next.

3.3 Version Control

Git was used for version control with the central repository hosted remotely on GitHub. This provided a remote backup of the codebase and a comprehensive audit trail of the system's architectural evolution.

3.4 Supervisor Meetings

To ensure the project remained aligned with both academic requirements and industry best practices, regular progress meetings were held with the project supervisor. These meetings acted as the equivalent of Agile "Sprint Reviews" since they took place towards the end of the week.

During these sessions, discussions that would take place would include last week's action items and how they've been addressed, this week's progress, decisions made within the project and their justifications, the action items for next week, and occasionally a demo of the project at that point in time.

3.5 User Stories

Within the Agile methodology, high-level system requirements were captured and documented as User Stories. To illustrate how the project can be applied in practice, this subsection presents three user stories that map directly onto solutions that this project aims to solve.

NOTE: The foundational user stories presented in this section were originally formed in the Semester 1 report for this project and have been retained as they remain the core operational use cases for the finalised architecture.

3.5.1 User Story 1

Scenario: A data platform team runs daily batch analytics jobs that are throughput-oriented but not latency-critical. They require these jobs to execute on the least carbon-intensive nodes available, while simultaneously bin-packing workloads efficiently to avoid provisioning additional underlying infrastructure.

Approach: Platform administrators define a declarative, namespace-scoped `WorkloadPolicy` that targets the batch jobs using standard Kubernetes label selectors that targets the batch jobs with the selected labels in the `WorkloadPolicy` itself. When the team deploys their jobs, a Mutating Admission Webhook intercepts the API request, automatically injecting the required sustainability metadata and carbon constraints in the pod annotations, subsequently routing the workloads directly to the `susk8s-scheduler`.

Outcome: The workloads are evaluated by the scheduler’s `CarbonBinPack` scoring plugin (and the `CarbonFilter` plugin if “hard” enforcement is enabled in the `WorkloadPolicy`), which successfully consolidates the jobs’ pods onto fewer, highly utilised nodes situated in low-carbon regions. This achieves improvements in both energy efficiency and emissions per run, enforcing the organisation’s sustainability intent without requiring any workflow changes from the engineering team.

3.5.2 User Story 2

Scenario: A platform engineering team supports multiple internal teams on a shared Kubernetes cluster. The organisation requires periodic sustainability reporting for internal governance and external audits, including energy usage and estimated carbon emissions by namespace.

Approach: The reporting process is driven entirely by the `EmissionReport` CRD, allowing administrators to define specific audit scopes and time windows. To populate this report, the system’s dedicated controller uses Kepler to capture container-level energy metrics directly from the kernel. It queries these hardware metrics from Prometheus and combines them with real-time grid carbon intensity data, materialising highly accurate, namespace-level emission totals directly into the report’s status field.

Outcome: The organisation can generate reproducible, auditable audits by applying an `EmissionReport` CRD. This provides sustainability accounting that eliminates the need for data-gathering scripts, while maintaining a strict architectural separation between telemetry collection, policy enforcement, and scheduling logic.

3.5.3 User Story 3

Scenario: A product team manages a highly available, long-lived API service distributed across multiple worker nodes. Over the workload’s lifecycle, the underlying grid carbon intensity of its host nodes fluctuates and change (e.g., carbon intensity increases or node power characteristics shift). The team wants workloads to remain compliant with a sustainability policy *after* initial placement, without sacrificing reliability.

Approach: The custom `ReschedulingController` continuously monitors the local API cache. If a node’s real-time carbon intensity exceeds the threshold defined in the governing `WorkloadPolicy`, the controller takes reactive or proactive action. Crucially, it does not abruptly terminate the non-compliant pods but instead, it issues a graceful termination request via the native Kubernetes Eviction API.

Outcome: By utilising the Eviction API, the system strictly respects `PodDisruptionBudgets`, ensuring that sustainability-driven rescheduling never compromises the application’s high availability. The evicted pods are naturally returned to the Pending queue, where the `susk8s-scheduler` re-evaluates and places them onto greener nodes. This allows the cluster to autonomously and safely converge toward optimal sustainability as external grid conditions shift.

4 System Architecture and Design

This section aims to explain the overall architecture of the system at a high level, covering the architectural evolution of the project along the way. The initial architecture covered in semester one is shown by the diagram in Figure 4:

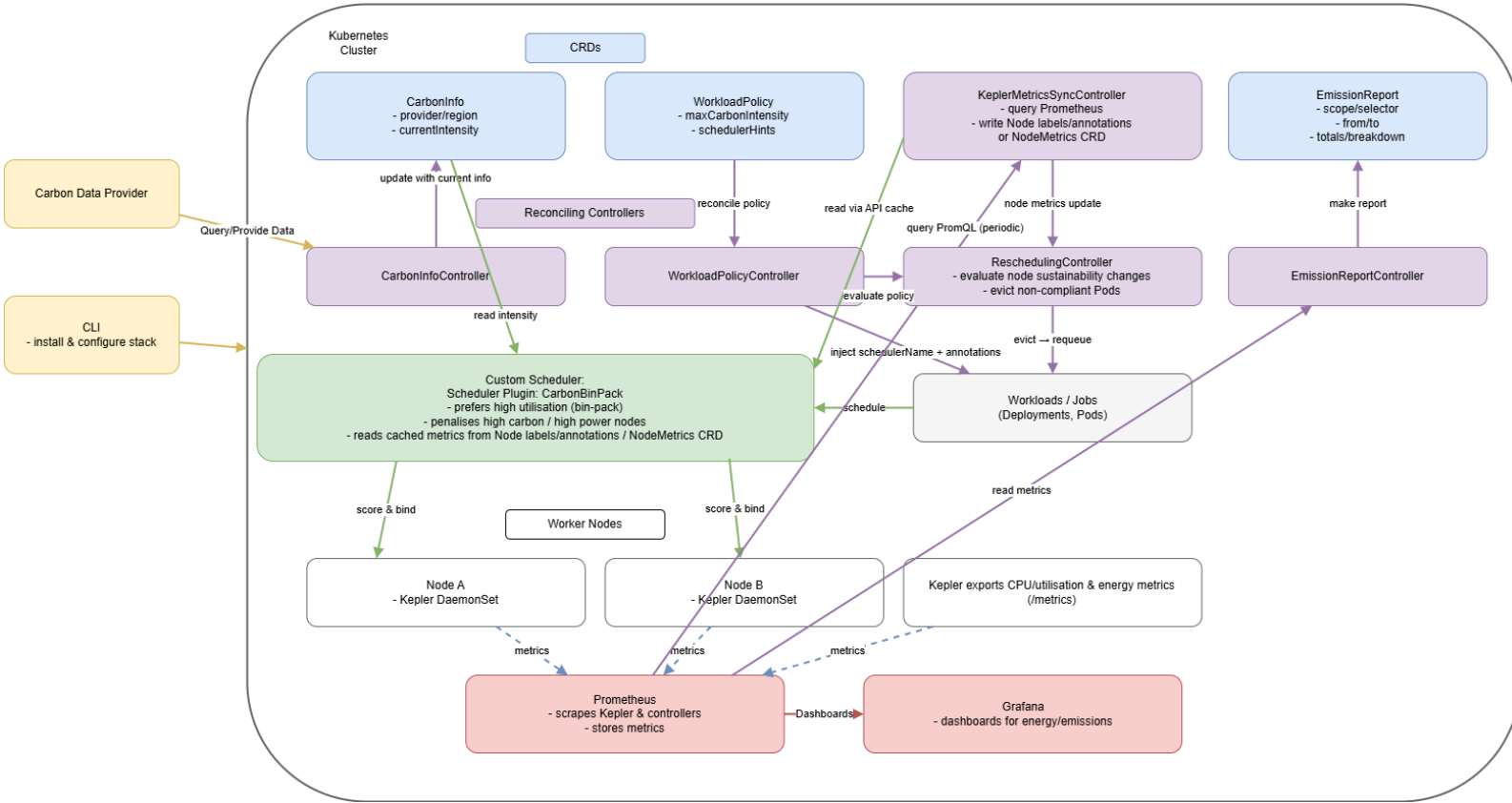


Figure 4: Initial architecture of the Sustainable Kubernetes Extension Stack.

This initial architecture included the following components:

- **External Inputs & Management**

- **Carbon Data Provider:** An external source queried to provide real-time carbon intensity data.
- **CLI:** A command-line interface used to install and configure the stack.

- **Custom Resource Definitions (CRDs)**

- **CarbonInfo:** Stores the region, provider and the current carbon intensity returned by said provider.
- **WorkloadPolicy:** Defines the maximum allowed carbon intensity and scheduler hints for specific workloads.
- **EmissionReport:** Defines the scope, selectors, and timeframes (from/to), and stores the calculated totals and breakdown of emissions.

- **Custom Operator and Reconciling Controllers**

- **CarbonInfoController:** Reads intensity data from the external provider and updates the CarbonInfo CRDs in the cluster.
- **WorkloadPolicyController:** Evaluates policies and directly injects the `schedulerName` and relevant annotations into the Workloads/Jobs (Deployments/Pods).

- **KeplerMetricsSyncController:** Periodically queries Prometheus and writes the data to Node labels/annotations via the local API cache.
- **ReschedulingController:** Evaluates node sustainability changes and safely evicts pods that are non-compliant with the rescheduling rules in the **WorkloadPolicy** that the pods match up with, forcing them back to the **Pending** state where they are to be requeued and reevaluated by the **susk8s-scheduler** scoring plugin.
- **EmissionReportController:** Reads workload metrics and compiles the final emission report.

- **Custom Scheduler**

- Features a custom scoring plugin called **CarbonBinPack**.
- Designed to prefer high node utilisation (bin-packing) while actively penalising high-carbon or high-power nodes.
- Makes scheduling decisions (scoring and binding) by reading the metrics projected onto the nodes, binding pods to the cleanest nodes.

- **Telemetry & Observability Pipeline**

- **Kepler DaemonSets:** Installed on the Worker Nodes of the cluster to export CPU, utilisation, and energy metrics.
- **Prometheus:** Scrapes and stores metrics from both the Kepler endpoints and the custom controllers.
- **Grafana:** Pulls data from Prometheus to display dashboards for cluster energy and emissions.

As the project developed in Semester 2, some new additions were added, as shown in [Figure 5](#)

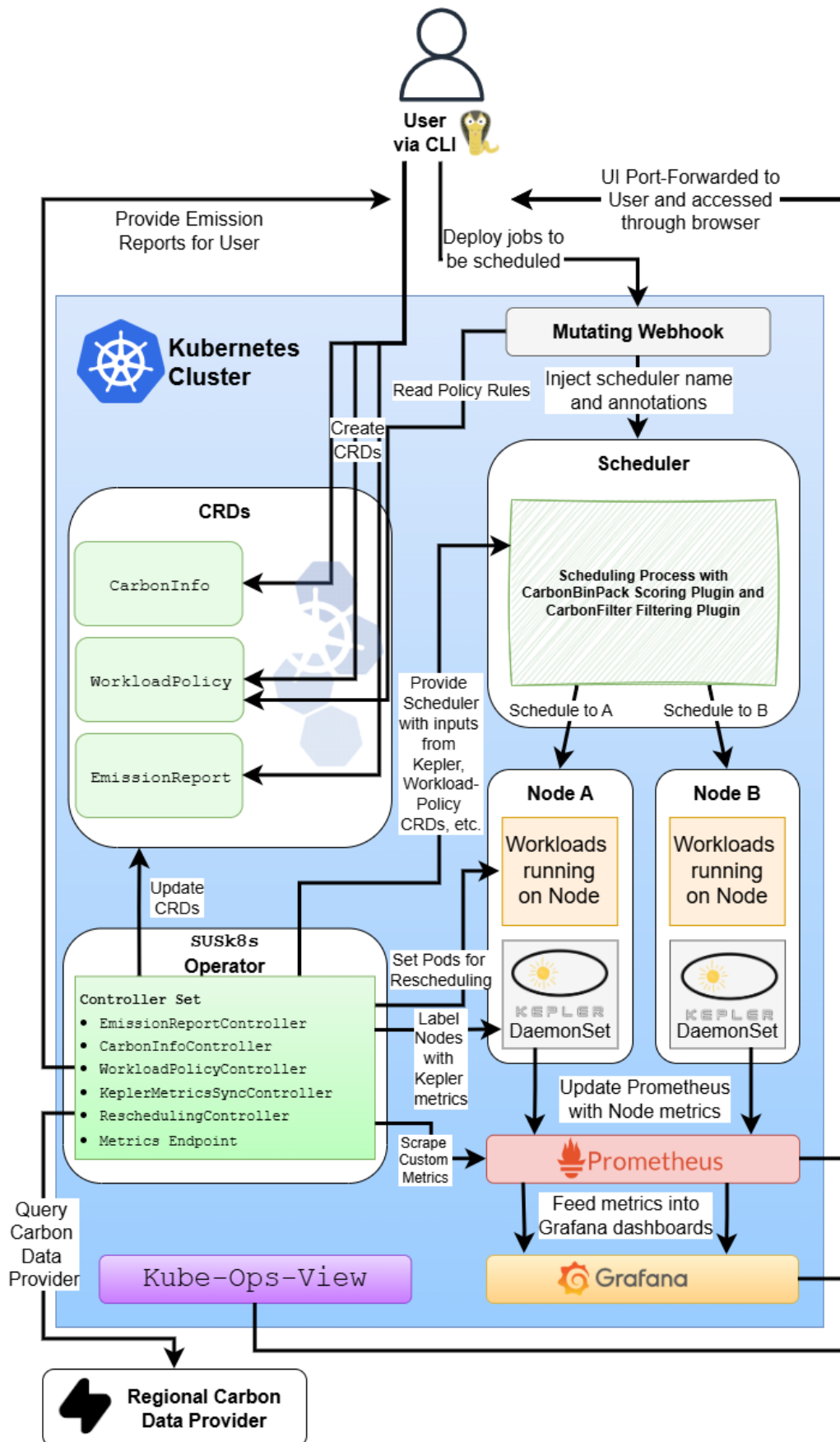


Figure 5: Current architecture of the Sustainable Kubernetes Extension Stack. (**NOTE:** The Mutating Admission Webhook runs inside of the custom operator but is detached from it here for clarity.

The updated architecture introduces the following major additions:

- **Automated Workload Interception via Mutating Webhook**

- This runs inside the custom operator intercepts incoming pods and checks for specific labels (e.g., `susk8s.io/tier`), acting as an automated gatekeeper during the deployment phase.
- It automatically mutates the pod in memory to inject the `susk8s-scheduler` and the specific carbon limits defined in the corresponding `WorkloadPolicy`, completely removing the manual configuration burden from developers.

- **CarbonFilter Plugin**

- Upgrades the custom scheduler from merely "scoring" nodes to actively filtering them.
- Compares the pod's requested carbon limit against the node's real-time grid intensity.
- Introduces dynamic enforcement modes: "hard" mode strictly blocks the pod from scheduling on dirty nodes, while "soft" mode allows it to proceed, giving teams control over the trade-off between sustainability and availability.

- **Operator Metrics Endpoint**

- The Operator was upgraded with a metrics endpoint for Prometheus to scrape.
- It calculates and exposes deep, real-time telemetry, including exact node energy usage (Joules), real-time cluster emissions (gCO₂/h), and total pod eviction counters.
- This data pipeline enables platform engineering teams to build live, highly accurate Grafana dashboards to monitor the immediate environmental impact of their infrastructure.

- **Kube-Ops-View**

- The system now integrates Kube-Ops-View to provide a real-time, graphical representation of the cluster's spatial state.
- Serves as a vital observability tool to visually validate the custom scheduler's logic, allowing administrators to monitor successful scheduling, as well as being able to observe pods migrating away from carbon-intensive nodes during automated evictions.

Having established the components of the architecture, these components will be covered at a lower level in greater detail in the following section.

5 Individual Components

Building upon the high-level system established in the previous section, this section deconstructs the components of the stack and looks at them at a lower level. The following subsections provide an in-depth examination of the stack's core elements, specifically detailing the CRDs, the custom operator's reconciling controllers, the Mutating Admission Webhook, and the custom scheduler and scheduling plugins.

5.1 CRDs

The CRDs in this system capture sustainability parameters and preferences as Kubernetes-native APIs. They include OpenAPI validation, configurable default values, and standard status conditions, and are scaffolded using Kubebuilder. Custom controllers reconcile these resources, projecting their outputs to the mutating webhook, the custom scheduler, and the observability stack.

These, alongside the operator and its reconciling controllers, were initialised using Kubebuilder.

Summary of CRDs

- **CarbonInfo**: Provides regional or node-group carbon intensity as a live signal for the cluster.
- **WorkloadPolicy**: Defines carbon-aware scheduling *and rescheduling* preferences and thresholds for labelled workloads.
- **EmissionReport**: Provides auditable energy and carbon emissions summaries over a reporting window.

CarbonInfo *Purpose*: To represent regional or node-group carbon intensity as a live signal.

Spec:

- **provider**: The data source supplying the intensity metrics (Electricitymaps or mock).
- **region**: The provider-specific geographic key for the grid data (IE, GB, etc.)
- **pollSeconds**: The time between refreshes for fetching live intensity data.
- **staticIntensity**: A fallback or fixed gCO₂/kWh value used for static mode or failed fetches.
- **overrideIntensity**: An optional manual override to force a specific intensity value (for developers, testers, etc.)

Status:

- **currentIntensity**: The latest fetched grid carbon intensity in gCO₂/kWh.
- **lastUpdated**: The timestamp of the last successful data retrieval from the provider.
- **conditions**: Represents the current reconciliation state of the carbon signal (e.g., **Ready**, error message if fetches are failing).

```
1 apiVersion: sustainability.susk8s/v1alpha1
2 kind: CarbonInfo
3 metadata:
4   creationTimestamp: "2026-05-07T13:22:48Z"
5   generation: 1
6   name: live-carbon-data3
7   namespace: default
8   resourceVersion: "6779"
9   uid: f1bc2839-cdb0-448c-8c1f-554238c6cb8a
10 spec:
11   pollSeconds: 60
12   provider: electricitymaps
13   region: AZ
14 status:
15   conditions:
16   - lastTransitionTime: "2026-05-07T13:22:48Z"
17     message: Successfully fetched intensity from electricitymaps
18     reason: Synced
19     status: "True"
20     type: Ready
21   currentIntensity: 363
22   lastUpdated: "2026-05-07T13:22:48Z"
```

Listing 1: Sample CarbonInfo YAML

WorkloadPolicy *Purpose:* To define the sustainability constraints governing both the initial pod placement (via the scheduler plugins) and Day 2 post-placement “rescheduling” or pod evictions.

Spec:

- **target:** Selects the workloads that fall under the policy using **matchLabels** within the deployment’s local namespace.
- **maxCarbonIntensity:** An upper bound (gCO₂/kWh) enforced by the scheduler’s **CarbonFilter** plugin to determine node eligibility.
- **schedulerHints:** Configures how the **CarbonBinPack** plugin should weigh initial placement, including node **utilisationWeight** and **carbonWeight**.
- **enforcement:** Controls whether the **maxCarbonIntensity** acts as a **soft** preference or a **hard** constraint.
- **reschedule:** Defines the rules for eviction-based rescheduling when node conditions change, including:
 - **enabled:** flag to toggle rescheduling,
 - **mode:** **soft** for only evicting when **maxCarbonIntensity** is surpassed or **hard** for proactively looking for cleaner nodes to reschedule to,
 - **cooldownSeconds:** minimum time between evictions,
 - **evictionRateLimit:** restricts the volume of pods evicted per reconciliation cycle,
 - **minImprovement:** defines the minimum amount of sustainability improvement required to justify an eviction.

NOTE: While defined in the CRD schema, the active enforcement logic for the **minImprovement** field was not finished due to project time constraints and remains a candidate for future work.

Status:

- **enforced:** A boolean showing whether scheduling and rescheduling hints are being successfully applied.
- **matchedWorkloads:** The current number of Deployments targeted by this specific policy.

NOTE: While standard Kubernetes **conditions** arrays are often used for detailed state reporting, they were omitted from this CRD’s implementation. The combination of the **enforced** boolean and the **matchedWorkloads** counter was evaluated to provide sufficient, immediate observability for policy reconciliation, avoiding the unnecessary overhead of maintaining verbose status arrays.

```

1  apiVersion: v1
2  items:
3  - apiVersion: sustainability.susk8s/v1alpha1
4    kind: WorkloadPolicy
5    metadata:
6      annotations:
7        kubectl.kubernetes.io/last-applied-configuration: |
8        {"apiVersion":"sustainability.susk8s/v1alpha1","kind":"WorkloadPolicy",
9          "metadata":{"annotations":{},"name":"demo-sustainability-policy",
10           "namespace":"default"},"spec":{"enforcement":"hard","
11           maxCarbonIntensity":200,"reschedule":{"cooldownSeconds":30,"
12           enabled":true,"evictionRateLimit":5,"mode":"hard"},"schedulerHints
13          ":{"carbonWeight":80,"utilisationWeight":20},"target":{"
14           matchLabels":{"susk8s.io/tier":"ultra-green"}}}}
15      creationTimestamp: "2026-05-07T13:12:43Z"
16      generation: 1
17      name: demo-sustainability-policy
18      namespace: default
19      resourceVersion: "5222"
20      uid: 9d684401-4eda-4728-a0c9-4c82d738216d
21    spec:
22      enforcement: hard
23      maxCarbonIntensity: 200
24      reschedule:
25        cooldownSeconds: 30
26        enabled: true
27        evictionRateLimit: 5
28        mode: hard
29      schedulerHints:
30        carbonWeight: 80
31        utilisationWeight: 20
32      target:
33        matchLabels:
34          susk8s.io/tier: ultra-green
35    status:
36      enforced: true

```

Listing 2: Sample CarbonInfo YAML

EmissionReport *Purpose:* To summarise cluster-wide or workload-level energy and emission metrics into auditable sustainability reports.

Spec:

- **scope:** Defines the level of data aggregation, accepting `namespace`, `workload` (pods), `deployment`, or `node`.
- **region:** Links the generated report to a specific grid carbon region.
- **from, to:** Timestamps defining the reporting window.

Status:

- **totals:** Contains the overall cluster-wide `energyKWh` and `emissionsGC02` for the requested window.
- **breakdown:** Provides a view of totals per individual entity of the given scope.
- **conditions:** Represents the state of report generation.

```

1 apiVersion: sustainability.susk8s/v1alpha1
2 kind: EmissionReport
3 metadata:
4   annotations:
5     creationTimestamp: "2026-05-07T15:30:53Z"
6     generation: 1
7     name: perfect-audit
8     namespace: default
9     resourceVersion: "3106"
10    uid: bde2aa68-f80e-4b5c-b2d7-2b0d93222a91
11 spec:
12   from: "2026-05-07T14:30:00Z"
13   region: GB
14   scope: namespace
15   to: "2026-05-07T15:30:00Z"
16 status:
17   breakdown:
18     unallocated_system_processes:
19       emissionsGC02: 14.745134780272966
20       energyKWh: 0.06118313186835256
21   conditions:
22   - lastTransitionTime: "2026-05-07T15:30:53Z"
23     message: Report generated using intensity 241 gCO2/kWh
24     reason: CalculationComplete
25     status: "True"
26     type: Finished
27   totals:
28     emissionsGC02: 8.614828691137546
29     energyKWh: 0.03574617714164957

```

Listing 3: Sample CarbonInfo YAML

Evolution of CRDs As covered in the notes earlier, the CRDs in this project have had a few changes since the initial project report in semester one. In the initial design, the CRDs were conceptualised as broad schemas for sustainability governance, with the expectation that the `WorkloadPolicyController` would directly handle the mutation of `Deployments` to inject sustainability hints. However, as the system architecture matured to prevent split-brain conditions and disruptive rolling restarts in the `WorkloadPolicy Controller` and Mutating Admission Webhook, the `WorkloadPolicy` evolved into a decoupled policy source. It now drives metadata and annotation injection via the Mutating Admission Webhook only (this change and the reason behind it will be covered in greater detail in the next section), while its status fields were streamlined in favour of easily interpretable metrics like `matchedWorkloads` and `enforced`. As previously noted, the `minImprovement` field in `reschedule` was left unfinished due to time constraints on the project. Finally, a new `scope` field was added to the `EmissionReport` CRD schema, which allows users to choose which resource they wish to view their reports on.

5.2 Operator Controller Logic

The `susk8s` operator relies on a suite of controller reconciliation loops that follow the operator reconciliation pattern to align the cluster’s physical state with declarative sustainability intent. These controllers observe the CRDs, executes operational logic, and publishes derived data to the Kubernetes API cache or metrics system.

CarbonInfo Controller

- **Purpose:** Watches `CarbonInfo` resources to maintain a live carbon intensity signal.
- **External I/O:** Queries carbon intensity from configured external data providers (e.g., ElectricityMaps) or mock endpoints.
- **Actions:** Updates the CRD status with the latest grid intensity and projects this regional intensity directly onto the cluster’s `Node` objects as annotations (e.g., `susk8s.io/carbon-intensity`).

WorkloadPolicy Controller

- **Purpose:** Watches `WorkloadPolicy` and `Deployment` resources to track policy coverage.
- **Actions:** Acts as a passive observer. It evaluates cluster workloads against policy label selectors to calculate coverage, updating the `status.matchedWorkloads` and `status.enforced` fields.
- **Architectural Note:** This controller explicitly delegates the actual mutation of pods (such as injecting `schedulerName` and `max-carbon` annotations) to the synchronous Mutating Admission Webhook which will be covered shortly. This separation of concerns prevents split-brain conditions and avoids triggering disruptive rolling restarts.

Rescheduling Controller

- **Purpose:** Correcting suboptimal placements as regional carbon intensity fluctuates.
- **Watches:** `WorkloadPolicy` `reschedule` constraints and `Node` updates (specifically fluctuating carbon intensity annotations).
- **Actions:**
 - Detects running pods whose underlying nodes no longer satisfy their policy constraints.
 - Executes eviction-based rescheduling for `hard` policies, strictly bound by administrator-defined `evictionRateLimits` and `cooldownSeconds`.

This controller does not directly migrate workloads. Instead, it issues graceful termination requests via the native Kubernetes Eviction API. This ensures full compliance with `PodDisruptionBudgets`, safely returning pods to the Pending queue^[12] where the custom scheduler can rebind them to greener nodes.

EmissionReport Controller

- **Purpose:** Watches `EmissionReport` resources to generate sustainability breakdowns.
- **Inputs:** Kepler metrics from Prometheus and `CarbonInfo` intensities.
- **Actions:** Evaluates the requested report scope (`namespace`, `deployment`, `workload`, or `node`) and dynamically constructs targeted PromQL queries to calculate gCO₂ emission breakdowns per kWh directly into the CRD `status`.

KeplerMetricsSync Controller

- **Purpose:** Synchronises raw sustainability metrics from Prometheus directly into the Kubernetes control plane.
- **Inputs:** Periodic Prometheus queries fetching Kepler-exported power metrics for cluster nodes.
- **Actions:** Acts as a lightweight, read-only metrics bridge by calculating real-time node carbon emissions by multiplying live Kepler power telemetry with the node’s cached grid intensity and exposing the calculated totals natively as custom Prometheus metrics (e.g., `susk8s_cluster_realtime_emissions`) through the metrics endpoint and through annotations (e.g. `susk8s.io/energy-usage`).

This provides a datasource for downstream observability dashboards while maintaining a strict separation between the scheduling and reporting planes.

Mutating Admission Webhook

- **Purpose:** Intercepts Pod creation requests synchronously to enforce “Day 0” sustainability policies before the resource is persisted to the cluster database (`etcd`).
- **Triggers:** API Server `AdmissionReview` requests for newly created Pods. It evaluates these incoming workloads against all active `WorkloadPolicy` CRDs in the namespace.
- **Actions:** If a Pod matches a policy’s label selector, the webhook dynamically mutates the payload in memory. It overrides the default `schedulerName` to force routing to the `susk8s-scheduler`, and injects necessary constraints (e.g., `max-carbon`, `enforcement` modes, and scheduler weights) directly into the Pod’s annotations.

By mutating the Pod on its way into the cluster, this guarantees that sustainability constraints are applied seamlessly, entirely avoiding the race conditions associated with retroactive controller-based mutation, such as with the previously attempted `Workload Policy Controller`.

Operator Data Flow Summary

1. **CarbonInfo Controller** queries external data providers, updates the `CarbonInfo` status, and projects the latest regional grid intensity directly onto `Node` annotations to provide the cluster with a live, low-latency environmental signal.
2. **Mutating Admission Webhook** intercepts incoming Pod creation requests. It evaluates them against active `WorkloadPolicy` CRDs and injects scheduling constraints (e.g., `schedulerName`, `max-carbon`) directly into the Pod payload before it is persisted to `etcd`, effectively handing the workload off to the scheduling plane.
3. **Custom Scheduler** (`susk8s-scheduler`) takes over the pending Pods. Its `CarbonFilter` and `CarbonBinPack` plugins read the webhook-injected Pod constraints and the `CarbonInfo` populated `Node` annotations to make carbon-aware placement decisions (which will be covered in greater detail in the next section).
4. **WorkloadPolicy Controller** acts as a continuous, passive observer. It reconciles deployed workloads against policy label selectors to calculate active coverage, updating the `matchedWorkloads` and `enforced` fields to provide real-time, “Day 1” observability into policy adoption across the cluster.
5. **Rescheduling Controller** monitors running Pods against real-time `Node` carbon fluctuations. If a `Node` breaches a `hard` policy threshold, it issues graceful termination requests via the Kubernetes Eviction API, safely returning the Pod to the pending queue for greener re-placement.
6. **KeplerMetricsSync Controller** fetches live power data from Prometheus, multiplies it by the `Node`’s carbon annotation, and exports real-time emission totals (e.g., `susk8s_cluster_realtime_emissions`) back to Prometheus through the metrics endpoint, providing instant dashboard visibility.

7. **EmissionReport Controller** time-windowed auditing by executing certain PromQL queries (e.g., isolating namespaces or deployments) against historical Kepler telemetry, materialising highly accurate energy and carbon breakdowns into **EmissionReport** status fields.

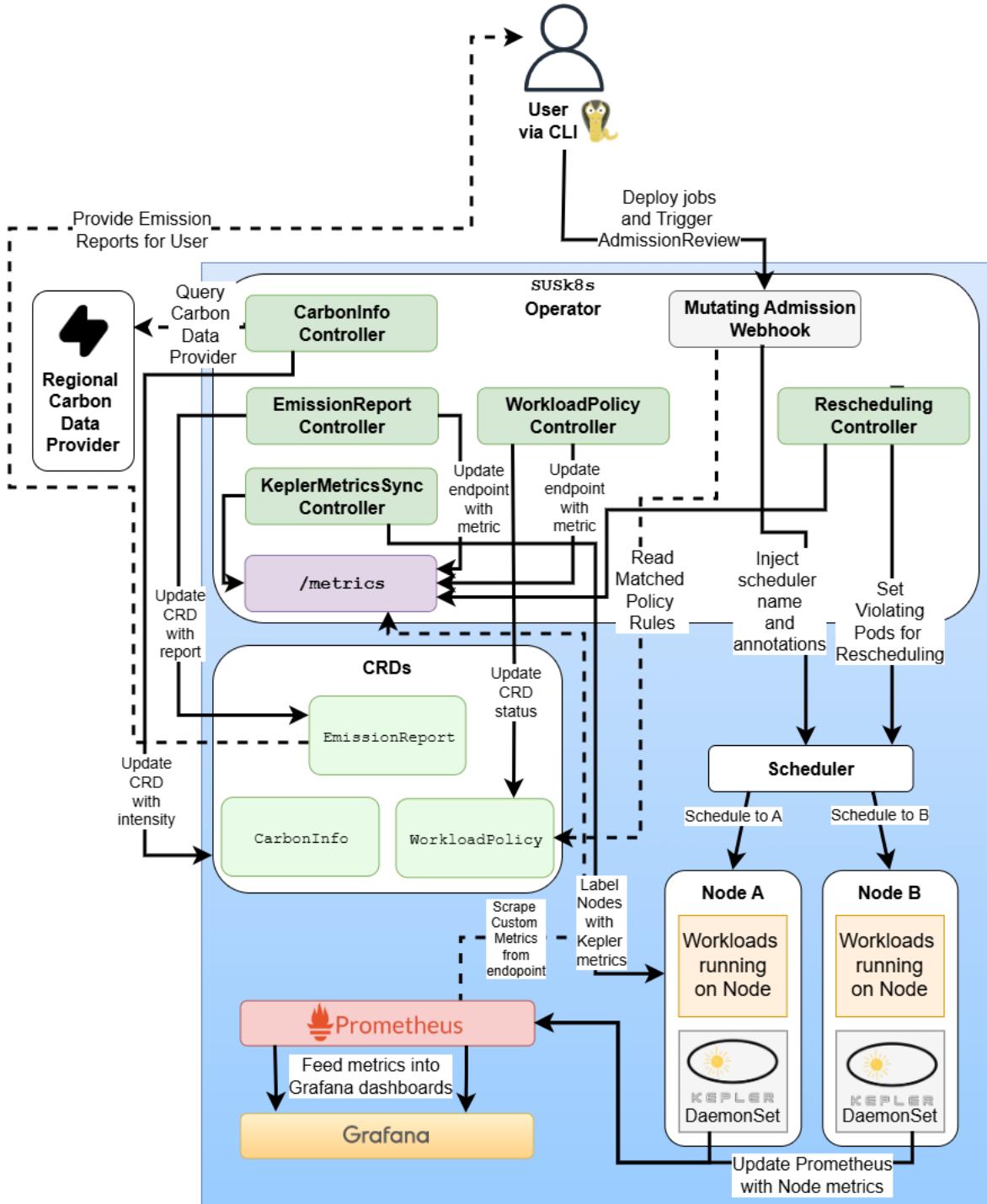


Figure 6: susk8s-operator diagram

Node-Level Sustainability Projection While sustainability intent and reporting are expressed via CRDs, node-level sustainability signals required for scheduling are projected directly onto Nodes using labels or annotations. This avoids introducing additional scheduler-facing APIs while allowing the controllers to materialise metrics for low-latency access during scheduling.

Evolution of custom operator Initially, the `CarbonInfo Controller`'s actions were to write to a cached `ConfigMap` mapping nodes or regions to intensity values for the scheduler plugin. The new annotation-based approach allows the custom scheduler plugins to read sustainability signals instantly from memory, avoiding the latency of querying external databases or `ConfigMaps` during scheduling.

As stated before, the responsibility for “Day 0” pod mutation was entirely stripped from the `WorkloadPolicy Controller` to eliminate race conditions and retroactive rolling restarts, and payload annotations (such as `max-carbon`) are now injected by the Mutating Admission Webhook before the resource is ever persisted to `etcd`.

The initial design of the `EmissionReport` CRD lacked targeting, generating long, cluster-wide sustainability summaries. By introducing the `scope` parameter into the CRD specification, the `EmissionReport Controller` was upgraded to support multi-tenant reporting. This evolution allows the operator to dynamically construct highly targeted PromQL queries on the fly, providing precise, workload-specific audits rather than relying on broad cluster-wide averages.

5.3 Scheduler Design and Implementation

As covered in the semester 1 report, the Kubernetes scheduler is responsible for assigning unscheduled Pods to available Nodes. The scheduler continuously watches for pending Pods and selects the best Node according to cluster policy to bind the Pod to [13].

5.3.1 Implementation Structure

The scheduler source code can be found in the `pkg/scheduler/` directory of the Kubernetes repository. The main interfaces are:

- **Filter:** decide whether a Node is feasible.
- **Score:** rank feasible Nodes with an integer 0-100.
- **PreBind / Bind / PostBind:** finalise scheduling.

Each phase is implemented through the *Scheduling Framework*, which provides plugin interfaces located in `pkg/scheduler/framework/`. This allows custom policies to be added without modifying core logic [15], allowing for the `CarbonBinPack` and `CarbonFilter` plugins to be seamlessly integrated into the custom scheduler.

5.3.2 Configuration and Extensibility

The scheduler is configured using a `KubeSchedulerConfiguration` object. This configuration-based design allows different workloads to use different scheduling policies within the same cluster.

To integrate the custom sustainability logic, a dedicated profile is created. This profile explicitly enables the custom plugins at their respective extension points within the Scheduling Framework, while inheriting the standard upstream behaviours for all other phases:

```
1 apiVersion: kubescheduler.config.k8s.io/v1
2 kind: KubeSchedulerConfiguration
3 profiles:
4   - schedulerName: susk8s-scheduler
5     plugins:
6       filter:
7         enabled:
8           - name: CarbonFilter
9       score:
10        enabled:
11          - name: CarbonBinPack
```


12

`weight: 100`

Listing 4: Sample KubeSchedulerConfiguration YAML

Once the profile is active, workloads can opt into this sustainability-aware placement by targeting the specific `schedulerName` in their Pod specification.

```

1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: sustainable-workload
5 spec:
6   schedulerName: susk8s-scheduler
7   containers:
8     - name: application
9       image: nginx:latest

```

Listing 5: Sample Pod YAML with `schedulerName`

While this project deploys a standalone custom scheduler binary (`susk8s-scheduler`) to ensure demonstrative clarity in carbon-aware scheduling, the plugins developed are compatible with the upstream `default-scheduler` and they can be integrated into any existing Kubernetes control plane simply by appending them to the default profile configuration.

5.3.3 Modified Scheduler Design for Sustainability

As discussed in previous sections, the default Kubernetes scheduler optimises Pod placement for resource availability and fairness but does not account for environmental impact. This project extends the scheduling process with carbon- and energy-aware placement by deploying a custom scheduler instance alongside the default scheduler, implemented using the Kubernetes Scheduling Framework to preserve upstream compatibility. Workloads are selectively routed to this scheduler through automatic injection of the `schedulerName` field by the Mutating Admission Webhook, enabling sustainability-aware scheduling without modifying application manifests or altering core Kubernetes components.

The primary objectives of the modified scheduler are:

- Reduce overall cluster energy consumption by consolidating workloads onto fewer, more efficiently utilised Nodes.
- Enable carbon-aware scheduling decisions using supplied sustainability metrics.
- Preserve full compatibility with upstream Kubernetes through the Scheduling Framework API.
- Allow opt-in behaviour through policy-driven configuration and webhook-driven payload mutation.

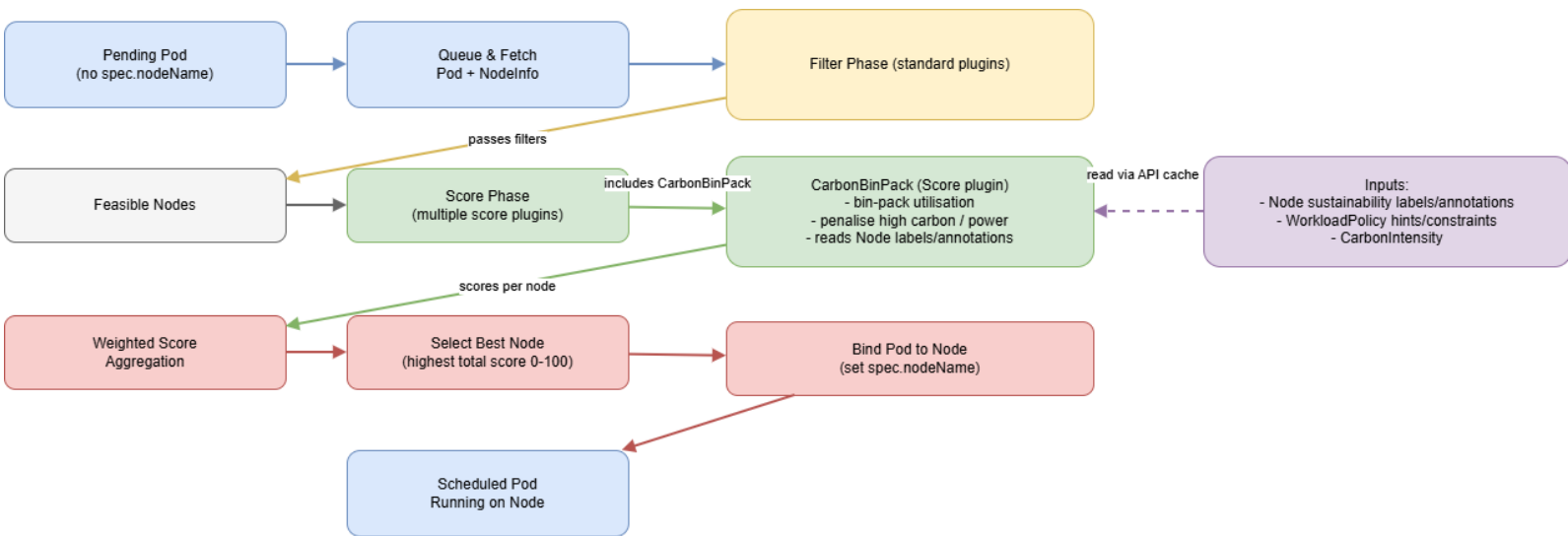


Figure 7: Kubernetes Scheduler flow, with **CarbonBinPack** scoring logic included

5.3.4 Evolution Scheduler Architecture

The initial scheduling pipeline consisted of the following components:

1. **WorkloadPolicy CRD** defining sustainability constraints and preferences.
2. **WorkloadPolicy Controller** that watches WorkloadPolicy resources and mutates matching Deployments by injecting sustainability-related annotations.
3. **Custom Scheduler** implemented as a standard `kube-scheduler` binary with additional plugins compiled in.
4. **CarbonBinPack scoring plugin** that evaluates Nodes during scheduling.
5. **Rescheduling Controller**, which performs eviction-based rescheduling, allowing affected Pods to be re-queued and placed according to current sustainability conditions.

This initial design presented significant operational limitations. Mutating Deployments via a standard controller introduced race conditions and triggered disruptive, unintended rolling restarts. Relying solely on a Score plugin meant the scheduler could express soft preferences for greener Nodes, but lacked the strict gating mechanisms required to enforce hard carbon emission limits.

To resolve these issues, the pipeline was fundamentally redesigned into the following configuration:

1. **Mutating Admission Webhook:** Intercepts incoming Pods and dynamically injects sustainability constraints (e.g., `max-carbon`) and the `susk8s-scheduler` routing directive directly into the payload.
2. **Custom Scheduler:** Same as initial implementation except now with the **CarbonFilter** now implemented.
3. **CarbonFilter Plugin:** Evaluates Nodes during the Filter phase, immediately rejecting Nodes whose real-time carbon intensity exceeds the Pod's `max-carbon` threshold.
4. **CarbonBinPack Plugin:** Evaluates Nodes during the Score phase, ranking them based on a combination of CPU utilisation and grid carbon intensity.
5. **Rescheduling Controller:** Acts as a Day 2 governor, continuously performing eviction-based rescheduling as grid intensities fluctuate.

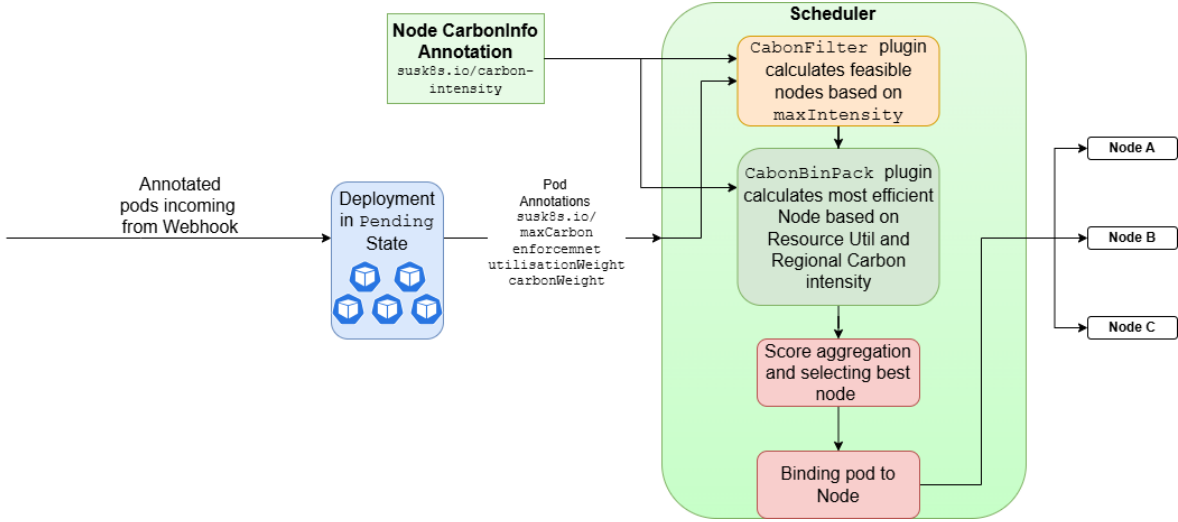


Figure 8: susk8s-scheduler flow.

5.3.5 Scheduler Plugin Implementation and Logic

The sustainability-aware logic is divided into two plugins which handle two different phases of the Scheduling Framework.

1. The Filter Phase (CarbonFilter): Before ranking occurs, the scheduler must determine if a Node is allowed to run the Pod. This phase reads the **enforcement** mode defined in the original policy and injected into the Pod payload by the webhook. The **CarbonFilter** reads the **max-carbon** and **enforcement** mode annotations to injected by the webhook to decide how nodes will be filtered. It compares the **max-carbon** against the **CarbonInfo** intensity projected onto the Node. If the Node's grid is currently too dirty, it is immediately filtered out of the candidate pool if the **enforcement** is set to "hard". If it is set to soft then the filter acts as a pass-through, leaving the placement decision entirely to the Score phase. To ensure visibility, the plugin simultaneously produces a native Kubernetes Event to the Pod object, warning users that the workload broke the soft filter conditions.

2. The Score Phase (CarbonBinPack): For the remaining feasible Nodes, the **CarbonBinPack** plugin executes to assign a relative score. The algorithm deliberately combines:

- **Resource Bin-Packing:** Prioritising already-active Nodes to maximise server utilisation and allow idle Nodes to scale down, thereby reducing baseline power draw.
- **Sustainability Signals:** Favouring Nodes located in regions with lower carbon intensity.

The final score is determined by the formula:

$$S_{final} = \frac{(S_{carbon} \times W_{carbon}) + (S_{util} \times W_{util})}{W_{carbon} + W_{util}}$$

The plugin retrieves the weights (W) from the Pod annotations `susk8s.io/utilisationWeight` and `susk8s.io/carbonWeight`, defaulting to 70 and 30 respectively. These weights can be any number, as opposed to the initial implementation where they both needed to add up to 100.

To calculate S_{carbon} , the plugin uses a maximum intensity threshold of 1000 gCO₂eq/kWh:

$$S_{carbon} = 100 - \left(\frac{I_{carbon} \times 100}{1000} \right)$$

For S_{util} , the plugin calculates the arithmetic mean of CPU and Memory utilisation based on the current requests in the **NodeInfo** snapshot:

$$S_{util} = \frac{\left(\frac{Req_{cpu}}{Alloc_{cpu}} \right) + \left(\frac{Req_{mem}}{Alloc_{mem}} \right)}{2} \times 100$$

Nodes that are both well-utilised and environmentally preferable receive the highest scores, guiding the scheduler toward highly energy-efficient placement decisions.

Initially, this scoring algorithm was designed to incorporate theoretical linear power models and hardware metrics from Kepler to calculate exact node power draw. However, evaluating complex power heuristics and making assumptions about P_{idle} and P_{max} during the scheduling cycle introduces inaccuracies in highly changing environment, as discussed in the linear power model section of the literature review.

5.4 Additional Tooling

While sustainability-aware scheduling and rescheduling define *where* workloads run, accurate telemetry is required to understand *why* placement decisions are made and to quantify their physical environmental impact. This project integrates existing observability components to measure power, energy, and emissions without introducing custom monitoring infrastructure.

5.4.1 Kepler Integration as the Measurement Layer

In this architecture, Kepler uses eBPF (Extended Berkeley Packet Filter) to attribute granular power consumption (in Joules) to specific Pods. Crucially, Kepler metrics are strictly decoupled from the scheduling plane and are **not** consumed directly by the scheduler. Instead, Kepler serves as the retrospective “ground truth” for cluster sustainability, providing the raw telemetry needed for Day 2 auditing and reporting.

5.4.2 Prometheus

Prometheus acts as the central time-series database and query engine for the SusK8s architecture. It aggregates the cluster’s environmental state by asynchronously scraping two primary targets:

- **Raw Hardware Telemetry:** The `/metrics` endpoint exposed by the Kepler `DaemonSet` on each Node.
- **Operator Telemetry:** The custom metrics endpoint exposed by the SusK8s Operator. This includes operational data, such as real-time cluster emissions (e.g., `susk8s_cluster_realtime.emissions`) calculated by the `KeplerMetricsSync Controller`, as well as counters for pod evictions.

By acting as the data store for the `EmissionReport Controller`, Prometheus enables the system to execute bounded, historical queries over defined time windows to compute highly accurate sustainability audits.

5.4.3 Grafana

Grafana consumes data exclusively from Prometheus, providing platform administrators with dynamic dashboards that expose real-time node energy consumption, spatial carbon intensity comparisons, and historical emission trends.

The custom panels include the following:

- **Regional Grid Carbon Intensity**
 - **Type:** Time-Series Graph
 - **Metric:** `susk8s_region_carbon_intensity_gco2_kwh`
 - **Purpose:** Provides a view of the carbon intensity for the broader geographical regions saved within the cluster. This serves as the baseline context for grid cleanliness before node-specific variations are applied.
- **Worker Node Carbon Intensity**
 - **Type:** Time-Series (Step Graph)
 - **Metric:** `susk8s_node_carbon_intensity_gco2_kwh`

- **Purpose:** Visualises the exact carbon intensity currently assigned to each individual worker node in real-time. This panel is crucial for tracking simulated weather events and verifying that the SusK8s Operator is correctly perceiving environmental changes across the cluster topology.
- **Live Node Energy Usage (Kepler)**
 - **Type:** Time-Series Graph
 - **Metric:** `susk8s_node_energy_usage_joules`
 - **Purpose:** Tracks the continuous energy consumption of each physical node. It translates deep eBPF hardware metrics into observable trends, highlighting how workload distribution physically impacts the underlying server hardware over time.
- **Carbon-Driven Pod Evictions (Last 5m)**
 - **Type:** Bar Chart / Time-Series
 - **Metric:** `increase(susk8s_pod_evictions_total[5m])`
 - **Purpose:** Acts as a real-time audit log for the autonomous system. It visually records the exact moments the Rescheduling Controller actively evicts pods due to `WorkloadPolicy` violations, verifying that the system’s enforcement mechanisms are functioning.
- **CPU vs RAM Power Draw (Cluster-Wide)**
 - **Type:** Donut / Pie Chart
 - **Metrics:** `kepler_node_core-joules-total` and `kepler_node_dram-joules-total`
 - **Purpose:** Breaks down the cluster’s total power consumption into its primary hardware components. This illustrates the varying energy demands of CPU-bound versus memory-bound operations during the experiment.
- **Most Power-Consuming Pods**
 - **Type:** Bar Gauge
 - **Metric:** `topk(5, sum by (pod_name) ...)`
 - **Purpose:** Identifies and ranks the top five most energy-intensive individual pods in the cluster. This provides administrators with immediate visibility into which specific workloads are driving the most significant power demands.
- **Live Total Cluster Power Draw**
 - **Type:** Stat Panel
 - **Metric:** `sum(rate(kepler_node_platform-joules-total...))`
 - **Purpose:** Displays a single, aggregated numerical value representing the absolute total power draw (in Watts) of the entire physical cluster infrastructure. It provides an immediate macroscopic view of the cluster’s real-time energy footprint.
- **Real-time Power Draw by Deployment**
 - **Type:** Bar Gauge
 - **Metric:** Aggregated `kepler_container_core-joules-total` by deployment label
 - **Purpose:** Groups the power consumption of individual pod replicas into their parent deployments. This is vital for understanding the total cost of an entire application or service, rather than just isolated pods.
- **True Savings**
 - **Type:** Time-Series Graph
 - **Purpose:** The definitive evaluation panel used for A/B testing. It dynamically plots four intersecting variables to prove the efficacy of the custom scheduler:

- * **Absolute Worst Case:** PromQL calculation simulating emissions if all workloads were placed on the highest-carbon node.
- * **Expected Vanilla Average:** PromQL calculation simulating the default Kubernetes scheduler's behavior (ignorant of carbon intensity). **NOTE: This line would generally way above the Kubernetes Vanilla runs in the A/B evaluation, which will be covered in more detail in the upcoming sections**
- * **True Actual Emissions:** The actual, real-world emissions of the SusK8s-managed cluster based on live telemetry (`susk8s_cluster_realtime_emissions_gco2.h`).
- * **Absolute Best Case:** PromQL calculation simulating the theoretical minimum emissions if idle power overhead was eliminated.

Custom SusK8s dashboards are applied to the cluster during the cluster creation through the `grafana-dashboard.yaml` ConfigMap manifest.

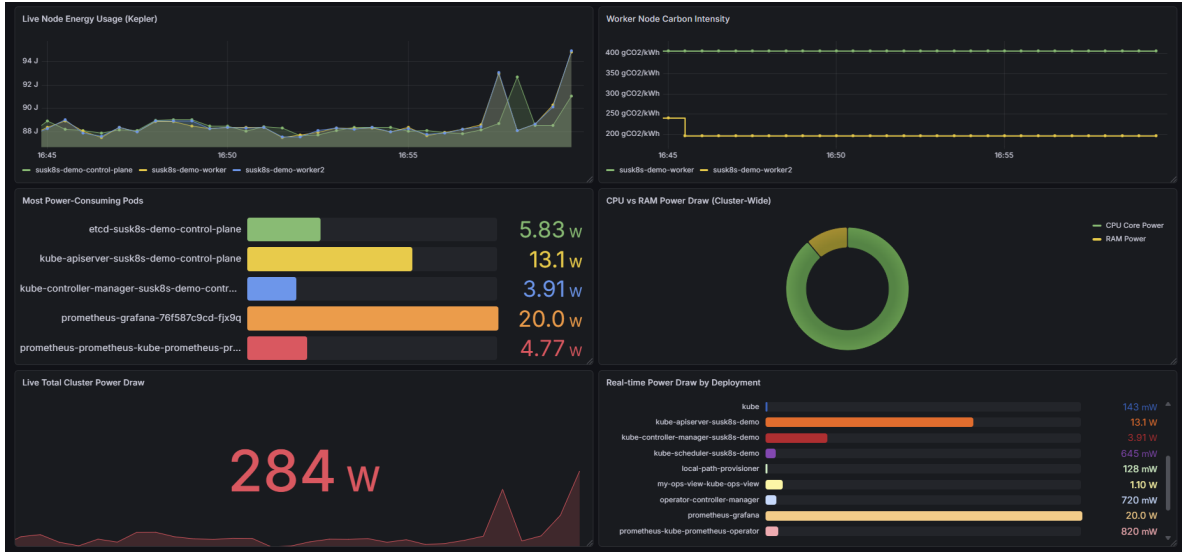


Figure 9: Custom SusK8s Grafana dashboards.

It must be noted that the Grafana dashboards provided in the Kepler Github repo [24] were also applied in previous iterations of the dashboard suite however, these dashboards were unreliable and prone to failure, which can be seen in Figures 10, 11 and 12.

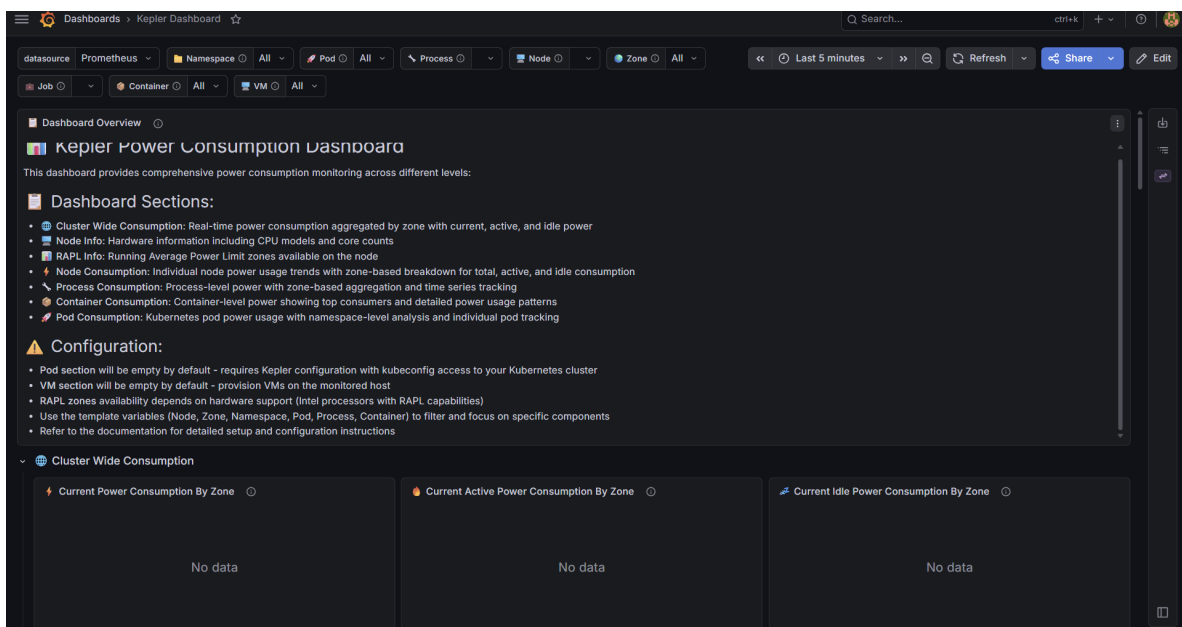


Figure 10: Kepler Grafana dashboards displaying no metrics 1.

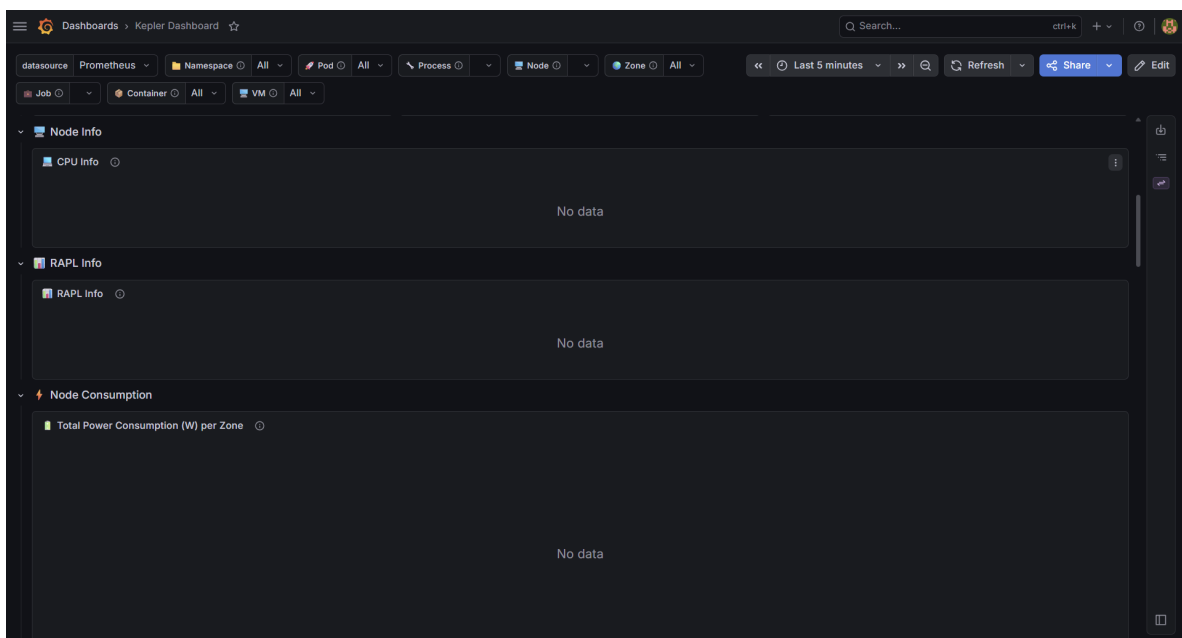


Figure 11: Kepler Grafana dashboards displaying no metrics 2.

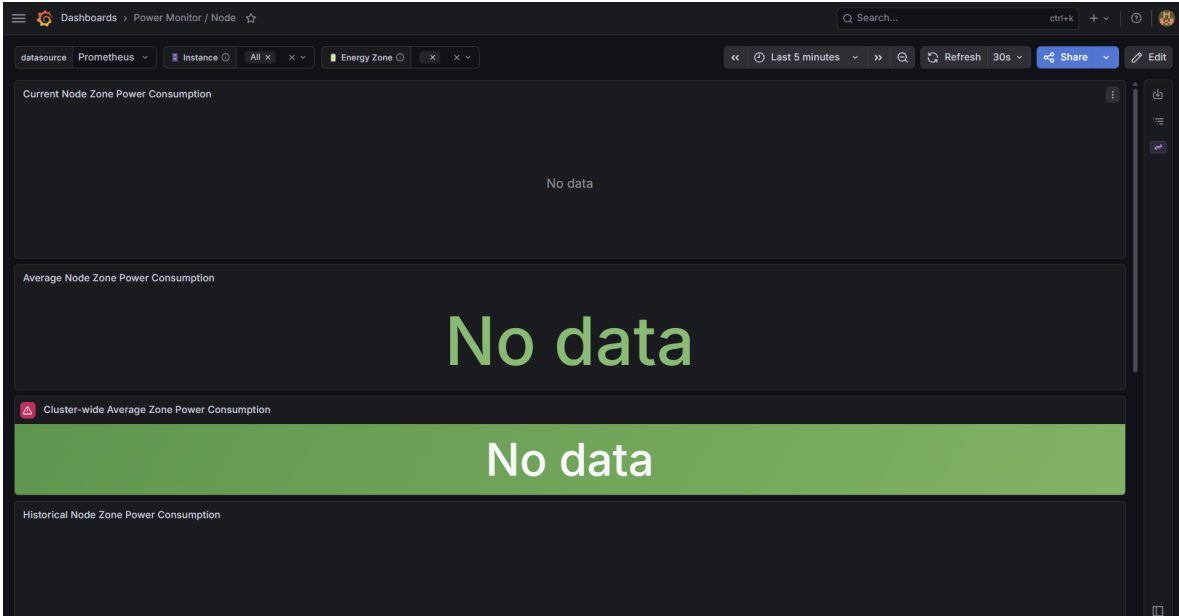


Figure 12: Kepler Grafana dashboards displaying no metrics 3.

Because of this, these dashboards had to be dropped.

5.4.4 Kube-Ops-View

Complementing the metrics dashboards, **kube-ops-view** is integrated to provide a live, topological visualisation of the cluster. This graphical interface maps node capacity and live pod distribution, serving as a critical observability tool to visually validate the **CarbonBinPack** logic and monitor the immediate spatial effects of pod evictions during periods of high grid carbon intensity.



Figure 13: Kube-Ops-View visualisation from the browser.

5.4.5 Cobra CLI

To bridge the gap between complex Kubernetes administration and user-friendly operations, the project includes a dedicated command-line interface built using the Cobra framework. The **susk8s** CLI abstracts the system's infrastructure into simple, reproducible commands, reducing configuration drift [26].

Core responsibilities of the CLI include:

- **Cluster Lifecycle Management:** Automating Day 0 provisioning of local development clusters (via Kind) and Day 2 installation of the entire SusK8s stack (CRDs, controllers, webhook, scheduler, and additional tooling) and deletion of the cluster.
- **Observability Routing:** Managing complex port-forwarding configurations with a single command to expose the Grafana, Prometheus, and Kube-Ops-View interfaces to the user.
- **Data and Auditing:** Allowing users to quickly fetch live **CarbonInfo** region data or trigger the generation of formatted **EmissionReport** receipts directly from the terminal.

6 Testing and Evaluation

Given the system’s distributed and controller-driven nature, testing is performed at multiple levels: unit, E2E, and a comparative A/B evaluation.

6.1 Scheduler Unit Testing

Unit tests are used to validate core logic in isolation, particularly where deterministic behaviour is expected. This includes:

- **Scheduler plugin logic:** scoring calculations to verify correct handling of utilisation, carbon weights, and boundary conditions.
- **Controller reconciliation logic:** individual reconcile loops are tested to ensure CRD condition handling and error paths behave as expected.
- **Validation rules:** API schema constraints on CRDs (e.g., weight ranges, required fields) are validated through negative test cases.

These tests are executed using Go’s standard **testing** framework and Kubernetes’ controller-runtime test utilities.

The custom scheduler plugins were rigorously unit-tested using the Kubernetes Scheduling Framework’s internal testing utilities, achieving over 90% code coverage (Figure 14).

- **CarbonFilter Logic:** Testing both **hard** and **soft** enforcement modes to ensure nodes are only rejected when strict limits are breached.
- **CarbonBinPack Math:** Simulating various CPU/Memory ratios and carbon intensities to ensure the scoring algorithm (S_{final}) correctly normalises non-standard weight inputs and accurately bounds the output score between 0 and 100.

```
unimu@gjangi:~/fyp/fyp/fyp/prototype/scheduler$ go test -coverprofile=coverage.out ./...
ok      github.com/georgelipceanu/fyp/prototype/scheduler/cmd/kube-scheduler      coverage: 0.0% of statements
ok      github.com/georgelipceanu/fyp/prototype/scheduler/pkg/framework/plugins/carbonbinpack 0.088s coverage: 100.0% of statements
ok      github.com/georgelipceanu/fyp/prototype/scheduler/pkg/framework/plugins/carbonfilter  0.094s coverage: 91.7% of statements
```

Figure 14: Unit tests passing on scheduling plugins with over 90% coverage.

6.2 Operator and Webhook Unit and Integration Tests

The operator controllers and mutating webhook were tested using the Ginkgo and Gomega Behavior-Driven Development frameworks, which were originally scaffolded by Kubebuilder after initialisation and expanded upon further after the fact. These frameworks utilise the **envtest** package to spin up a local control plane, allowing the controllers to interact with a live API server. This suite represents a hybrid testing approach: it performs integration testing by utilising the **envtest** package to spin up a control plane, ensuring controllers can successfully GET, LIST, and PATCH real resources. Simultaneously, the suite performs unit testing by isolating logic from external, non-Kubernetes dependencies. By mocking the Prometheus API and external carbon providers, these tests validate the operator’s internal state machine without requiring a full cluster or internet connectivity.

The combined test coverage of 71%. Testing focused heavily on validating state transitions and handling external dependency failures gracefully. Key test scenarios include:

- **Mutating Admission Webhook:** Validating that incoming Pod payloads are correctly intercepted, and that the appropriate `schedulerName`, `max-carbon` limits, and scaling weights are safely injected into the Pod annotations, while gracefully rejecting malformed JSON requests.
- **Prometheus Dependency Mocking:** Utilising custom mock interfaces (e.g., `MockPromAPI`) to verify that the `EmissionReport` and `KeplerMetricsSync` controllers correctly fallback to safety thresholds or gracefully skip operations when raw hardware telemetry is unavailable.
- **Passive Reconciliation:** Verifying that the `WorkloadPolicy Controller` accurately increments the `matchedWorkloads` counter for targeted deployments while safely ignoring non-matching workloads and malformed label selectors.
- **External API Resilience and Overrides:** Validating that the `CarbonInfo Controller` correctly prioritises manual intensity overrides and seamlessly degrades to static fallback values when external provider APIs fail, ensuring uninterrupted projection of carbon telemetry onto Node annotations.
- **Dynamic Rescheduling and Safe Eviction:** Validating that the `Rescheduling Controller` successfully issues graceful termination requests (simulating the native Eviction API) for pods violating `hard` carbon constraints, while strictly bypassing eviction when policies are configured for `soft` enforcement.

```

unimu@gjangi:~/fyp/fyp/fyp/prototype/operator$ make test
/home/unimu/fyp/fyp/fyp/prototype/operator/bin/controller-gen rbac:roleName=manager-role crd:allowDangerousTypes=true webhook paths="./.
.." output:crd:artifacts:config=config/crd/bases
/home/unimu/fyp/fyp/fyp/prototype/operator/bin/controller-gen object:headerFile="hack/boilerplate.go.txt" paths="./..."
go fmt ./...
go vet ./...
Setting up envtest binaries for Kubernetes version 1.33...
/home/unimu/fyp/fyp/fyp/prototype/operator/bin/k8s/1.33.0-linux-amd64KUBE_BUILDER_ASSETS="/home/unimu/fyp/fyp/fyp/prototype/operator/bin/
k8s/1.33.0-linux-amd64" go test $(go list ./... | grep -v /e2e) -coverprofile cover.out
    prototype/operator/api/v1alpha1      coverage: 0.0% of statements
    prototype/operator/cmd                coverage: 0.0% of statements
ok   prototype/operator/controllers      21.049s coverage: 71.6% of statements
ok   prototype/operator/internal/controller 17.319s coverage: 21.5% of statements
    prototype/operator/test/utils         coverage: 0.0% of statements

```

Figure 15: Unit tests passing on operator controllers with over 70% coverage. (**NOTE:** The files in `internal/controller` aren't actually used and are residual files which were not removed due to concerns with potentially breaking something or needing those files once again).

6.3 End-to-End Evaluation

To evaluate the full system workflow under realistic conditions, E2E testing was conducted using a local, multi-node KinD cluster. This phase tested the interaction of all components operating concurrently by applying YAML manifests and asserting against live cluster state via `kubectl` commands.

The E2E suite validated the system's dynamic scheduling and eviction capabilities through a set of different scenarios:

- **Dynamic Routing and Eviction:** Validates the full Day 0 to Day 2 lifecycle. It confirms that the Mutating Admission Webhook successfully intercepts the workload and injects the custom scheduler directives. The test then simulates a carbon spike on the host node, verifying via operator logs that the `Rescheduling Controller` successfully detects the violation and issues an eviction request.
- **The “Brownout” Scenario:** Simulates a severe grid carbon spike across all available worker nodes. This proves that the custom scheduler's `CarbonFilter` plugin correctly evaluates the nodes against the policy's `maxCarbonIntensity` and prevents the workload from scheduling, holding it safely in a `Pending` state rather than violating the environmental constraint.
- **Node Recovery (The “Sun Comes Out” Scenario):** Expanding on the brownout scenario, this test validates the scheduler's event-driven recovery mechanics. After a pod is intentionally held in a `Pending` state due to a grid-wide carbon spike, a single node's intensity is manually lowered via annotations to simulate changing weather conditions (e.g., increased solar or

wind generation). The test verifies that the custom scheduler successfully detects the improved conditions and immediately binds the queued workload to the green node.

- **Multi-Tiered Policy Enforcement:** Deploys concurrent “Platinum” (high limit: 500 gCO₂eq/kWh) and “Bronze” (strict limit: 150 gCO₂eq/kWh) workloads. The grid intensity is artificially spiked to a moderate level (300 gCO₂eq/kWh) to verify the precision of the **Rescheduling Controller**. The test asserts that the operator exclusively evicts the non-compliant Bronze workload while successfully preserving the high availability of the compliant Platinum service.

```
Ran 5 of 5 Specs in 234.609 seconds
SUCCESS! -- 5 Passed | 0 Failed | 0 Pending | 0 Skipped
--- PASS: TestE2E (234.61s)
PASS
ok      prototype/operator/test/e2e      234.638s
make cleanup-test-e2e
make[1]: Entering directory '/home/unimu/fyp/fyp/prototype/operator'
Deleting cluster "operator-test-e2e" ...
Deleted nodes: ["operator-test-e2e-worker2" "operator-test-e2e-control-plane" "operator-test-e2e-worker"]
make[1]: Leaving directory '/home/unimu/fyp/fyp/prototype/operator'
```

Figure 16: E2E tests passing

6.4 SusK8s vs Vanilla Kubernetes A/B Evaluation

To evaluate the system’s actual , a comparative A/B test was conducted against the default Kubernetes **kube-scheduler** using a 15-minute Brownout scenario. This experiment simulated severe grid fluctuations across a three-node cluster, artificially spiking Node 1 to a carbon intensity of 800 gCO₂eq/kWh while dropping Node 2 to 10 gCO₂eq/kWh. Node utilisation and physical energy consumption were continuously tracked via custom Grafana dashboards consuming Prometheus and Kepler eBPF telemetry, providing real-time visibility into spatial grid intensity, hardware energy, pod evictions, and emissions.



Figure 17: Grafana time-series graph of current emissions being produced by the worker nodes during Vanilla Kubernetes evaluation.

The data revealed that while the SusK8s operator successfully executed carbon-aware workload routing to evacuate the high-carbon node, it did not generate a net reduction in total cluster emissions, highlighting one of the main constraints of software-defined sustainability.

Over the 15-minute experiment over 5 runs, the unoptimised Vanilla cluster averaged 121.73 g/h, while the optimised SusK8s cluster averaged a statistically similar 122.22 g/h. On top of this, during peak load shifting, the SusK8s cluster recorded a higher maximum emission spike (184.00 g/h) compared to the baseline (164.00 g/h).

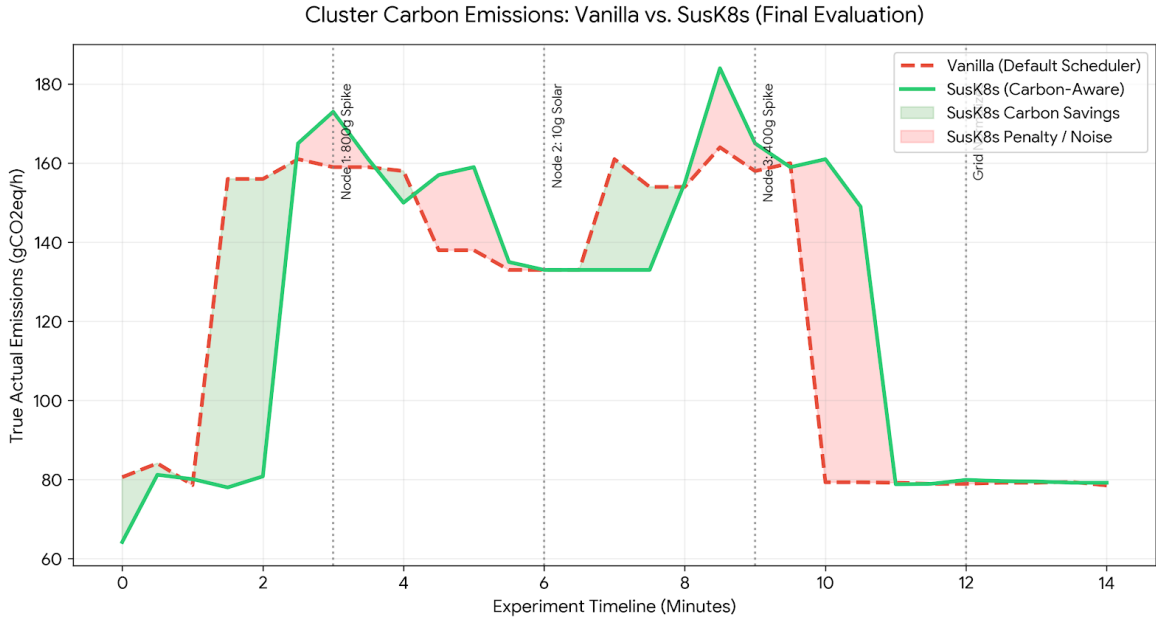


Figure 18: Cluster Carbon Emissions: Vanilla kube-scheduler vs. SusK8s Operator during the 15-minute Brownout experiment.

The data highlights two critical physical constraints that subvert software-only sustainability:

1. When SusK8s aggressively bin-packed all evacuated workloads onto the cleanest node (Node 2), that target server experienced a severe spike in CPU utilisation. Modern processors require an exponential increase in voltage to sustain peak clock frequencies. Consequently, the raw power draw (Watts) of the single overloaded node drastically increased, mathematically offsetting the benefit of the cleaner grid multiplier.
2. In a Kubernetes topology, an evacuated node is never truly idle. While user workloads were successfully removed from the 800g node, Daemonsets such as `kube-proxy`, CNI plugins, and the `kubelet` agent itself, all critical components to a Node's infrastructure, remained strictly bound to the hardware. These immovable system processes, combined with the baseline power draw of the physical server, maintained a continuous background draw which, when multiplied by the extreme 800g carbon intensity, generated an unavoidable emissions floor.

The data from the A/B evaluations proves the physical limitations of software-centric sustainability. The evaluation demonstrates that SusK8s successfully achieved application-level sustainability by moving specific software workloads from dirty power grids. However, workload-level scheduling is not enough alone.

To achieve true zero-carbon operations, intelligent schedulers must act as trigger mechanisms for infrastructure-level scaling. One thing with a Carbon-Aware Cluster Autoscaler, providing the necessary administrative authority to physically power down bare, metal servers after the software controller has successfully evacuated them.

7 System Deployment and User Guide

This section provides a practical guide for building, deploying, and interacting with the extension stack. The source is compartmentalised into three primary directories: the `operator`, the `scheduler`, the `demo-yamls`, and the `cli` directory. The following instructions detail the prerequisites and the manual build processes required to deploy the system from source.

7.1 Prerequisites

Before building the components, the host machine must have the following development dependencies installed:

- **Go (v1.21+)**: Required for compiling the operator, custom scheduler, and CLI binaries.
- **Docker**: Required for building the container images.
- **Kubernetes CLI (kubectl)**: Required for interacting with the cluster API.
- **Helm**: Required for installing the third-party observability charts (Prometheus, Kepler, etc.).
- **KinD (Kubernetes IN Docker)**: Recommended for local testing and development environments.
- **Make**: Required for executing the operator's Kubebuilder scaffolded build and deployment targets.

7.2 Deploying the Observability and Telemetry Stack

Before deploying the custom SusK8s components, the underlying observability infrastructure must be provisioned. This includes the certificate manager required by the Mutating Webhook, as well as the Prometheus and Kepler monitoring stack. These are the steps to take to install these in a cluster (provided that the cluster is running already)

1. **Install Cert-Manager**: Required to automatically provision TLS certificates for the Operator's admission webhooks:

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.14.4/cert-manager.yaml
```

2. **Add Helm Repositories**: Add and update the necessary third-party Helm repositories:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add kepler https://sustainable-computing-io.github.io/kepler-helm-chart
helm repo add christianhuth https://charts.christianhuth.de
helm repo update
```

3. **Install Prometheus and Kube-Ops-View**: Deploy the kube-prometheus-stack and Kube-Ops-View into the cluster:

```
helm upgrade --install prometheus prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace \
  --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false
```

```
helm upgrade --install my-ops-view christianhuth/kube-ops-view
```

4. **Install Kepler and Custom Dashboards**: Deploy Kepler with KinD-specific configurations, apply its required ServiceMonitors, and load the custom SusK8s Grafana dashboard:

```
helm upgrade --install kepler kepler/kepler \
  --namespace kepler --create-namespace \
  --set provider=kind
kubectl apply -f ../demo-yamls/kepler-svc.yaml
kubectl apply -f ../demo-yamls/kepler-sm.yaml
kubectl apply -f ../demo-yamls/grafana-dashboard.yaml
```

7.3 Deploying the SusK8s Operator and CRDs

The `operator` directory contains the Kubebuilder-scaffolded control plane, including the custom controllers, the Mutating Admission Webhook, and the CRD definitions. These are the steps to take to install the operator in a cluster (provided that the cluster is running already).

1. **Generate Manifests:** Navigate to the `operator` directory and generate the latest CRD and RBAC YAML manifests using the provided Makefile:

```
cd operator
make generate
make manifests
```

2. **Build and Load the Image:** Compile the operator binary, package it into a Docker image using the Kubebuilder Dockerfile, and load it directly into the local KinD cluster:

```
make docker-build IMG=susk8s-operator:latest
kind load docker-image susk8s-operator:latest --name <cluster-name>
```

3. **Deploy to the Cluster:** Install the CRDs and deploy the operator components into the `operator-system` namespace, and apply the custom ServiceMonitor so Prometheus can scrape its metrics:

```
make install
make deploy IMG=susk8s-operator:latest
kubectl apply -f ../demo-yamls/susk8s-monitor.yaml
```

4. **Apply Eviction RBAC Permissions:** To allow the Rescheduling Controller to safely issue pod eviction requests across all namespaces, the operator requires elevated cluster permissions. Apply the manual RBAC bindings and restart the controller to load the new security context:

```
kubectl apply -f ../demo-yamls/rescheduling-rbac.yaml

kubectl create clusterrolebinding operator-admin-bypass \
  --clusterrole=cluster-admin \
  --serviceaccount=operator-system:operator-controller-manager

kubectl rollout restart deployment/operator-controller-manager \
  -n operator-system
```

7.4 Deploying the Custom Scheduler

The `scheduler` directory contains the custom Kubernetes Scheduling Framework implementation, including the `CarbonFilter` and `CarbonBinPack` plugins.

1. **Build the Scheduler Image:** Navigate to the `scheduler` directory, compile the Go binary (disabling CGO for static linking), and build the Docker image:

```
cd scheduler
go mod download
CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -o bin/kube-scheduler-linux \
  ./cmd/kube-scheduler/main.go
docker build -t susk8s-scheduler:latest .
kind load docker-image susk8s-scheduler:latest --name <cluster-name>
```

2. **Apply the Configuration:** The scheduler requires a `KubeSchedulerConfiguration` to activate the custom plugins. Create a ConfigMap in the cluster containing this profile:

```
kubectl create configmap susk8s-scheduler-config -n kube-system \
  --from-file=scheduler-config.yaml=../demo-yamls/scheduler-config.yaml \
  --dry-run=client -o yaml | kubectl apply -f -
```

3. **Deploy the Scheduler:** Apply the scheduler Deployment manifest to run it as an independent, secondary scheduler alongside the default `kube-scheduler`:

```
kubectl apply -f ../demo-yamls/scheduler-deploy.yaml
```

7.5 Building and Using the CLI

The `cli` directory contains the Cobra-based `susk8s` command-line tool. While the steps above detail the manual developer deployment process, the CLI was engineered to entirely abstract these workflows for end-users, automating Day-0 cluster creation and Day-2 observability routing.

1. **Compile the Binary:** Navigate to the CLI directory and build the executable:

```
cd cli
go mod download
go build -o susk8s main.go
```

2. **System Installation (Optional):** Move the binary to the system path for global execution:

```
sudo mv susk8s /usr/local/bin/
```

3. **Core Commands:** Once installed, platform engineers can interact with the system using the following commands:

- **susk8s create:** Automates the provisioning of an isolated KinD cluster, generates a dedicated `kubeconfig`, and immediately triggers the installation routine.
- **susk8s install:** Installs the complete prerequisite observability stack (Cert-Manager, Prometheus, Kepler, Kube-Ops-View) alongside the custom operator, scheduler, and custom Grafana dashboards.
- **susk8s ui:** Safely waits for UI components to become ready, then automatically establishes background port-forwarding tunnels. This grants immediate localhost access to the Grafana (port 3000), Prometheus (port 9090), and Kube-Ops-View (port 8080) dashboards.
- **susk8s carbon:** Generates and applies a `CarbonInfo` CRD to stream live grid intensity data. It includes built-in developer flags (e.g., `--override`) to instantly simulate severe grid carbon spikes for testing Day-2 eviction logic.
- **susk8s report:** Dynamically generates and applies a time-bounded `EmissionReport` CRD. Users can pass flags for specific time windows and scopes (e.g., `--scope=deployment`) to calculate precise, multi-tenant energy and carbon audits.
- **susk8s delete:** Safely tears down the environment by gracefully uninstalling Helm releases, destroying the KinD virtual nodes, and cleaning up local credential files.

7.6 Video Demo

A video demo of the stack in action is available to watch in the video demo submission box of the Final Year Project module on Moodle.

8 Future Work

While the SusK8s project in its current state successfully demonstrates the viability of decoupled, carbon-aware scheduling and observability, several avenues remain for extending the platform’s security, scalability, and feature set.

8.1 Carbon-Aware Autoscaler

As demonstrated in the empirical A/B evaluation, achieving application-level sustainability through workload eviction and carbon weighting is not enough to eliminate a cluster’s carbon footprint due to the unavoidable idle power penalty of bare-metal hardware. To achieve true infrastructure-level sustainability, future iterations of SusK8s must integrate a Carbon-Aware Cluster Autoscaler. By integrating the SusK8s operator with existing elasticity tools, such as the native Kubernetes Cluster Autoscaler or Karpenter, SusK8s could serve as a trigger mechanism for infrastructure scaling.

8.2 Multi-Region Cluster Routing

The current `CarbonFilter` and `CarbonBinPack` plugins operate exclusively at the intra-cluster level, routing pods to the greenest available node within a cluster, with a nodes region depending on the region annotation placed on it, effectively making the current iteration of this project a demo of sustainable multi-region cloud deployments. Future research for this project would include extending this logic to the inter-cluster level.

By integrating cluster orchestration solutions that spread across many regions, such as Karmada or even EKS Multi-Zone Clusters, a global control plane could be established. This would allow a global load balancer to dynamically route incoming workloads to entirely different geographic regions (e.g., shifting traffic from IE to AZ) based on real-time solar or wind availability, effectively creating a globally shifting, sustainable cloud architecture.

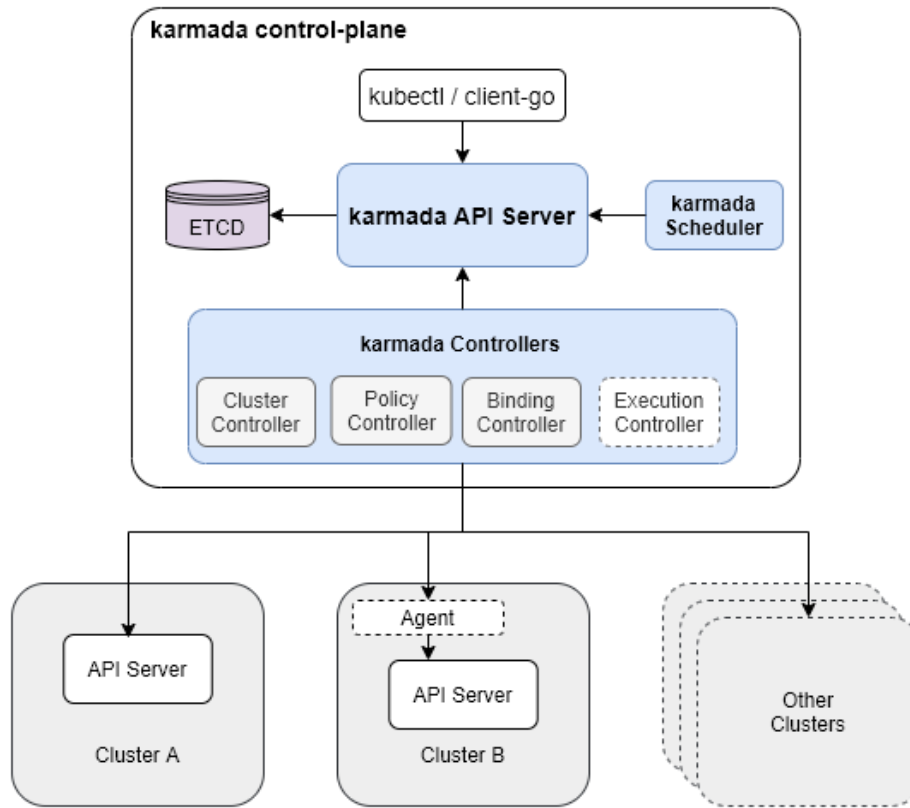


Figure 19: Karmada system architecture diagram, showing how the global control plane interacts with the multi-cluster deployment [9].

8.3 Additional Observability Options

The current observability stack provides decent insights into the sustainability of the cluster however due to time constraints, there was a lot that had to be cut from the project. The current iteration of this project provides a good foundation of for observability but future iterations should expand on this foundation. Potential additions include:

- **Historical Multi-Tenant Accountability:** While the current dashboard displays real-time power draw per deployment, it relies on Prometheus metrics which are relatively short-term, defaulting to 15 days worth of metrics before they are deleted. Future work would involve persisting the time-bounded **EmissionReport** CRD outputs into long-term metric storage, such as Thanos. This would enable the creation of historical, namespace-level dashboards, allowing individual development teams to track their carbon footprint over months or years without taking up the disk space that it would take to do with Prometheus.

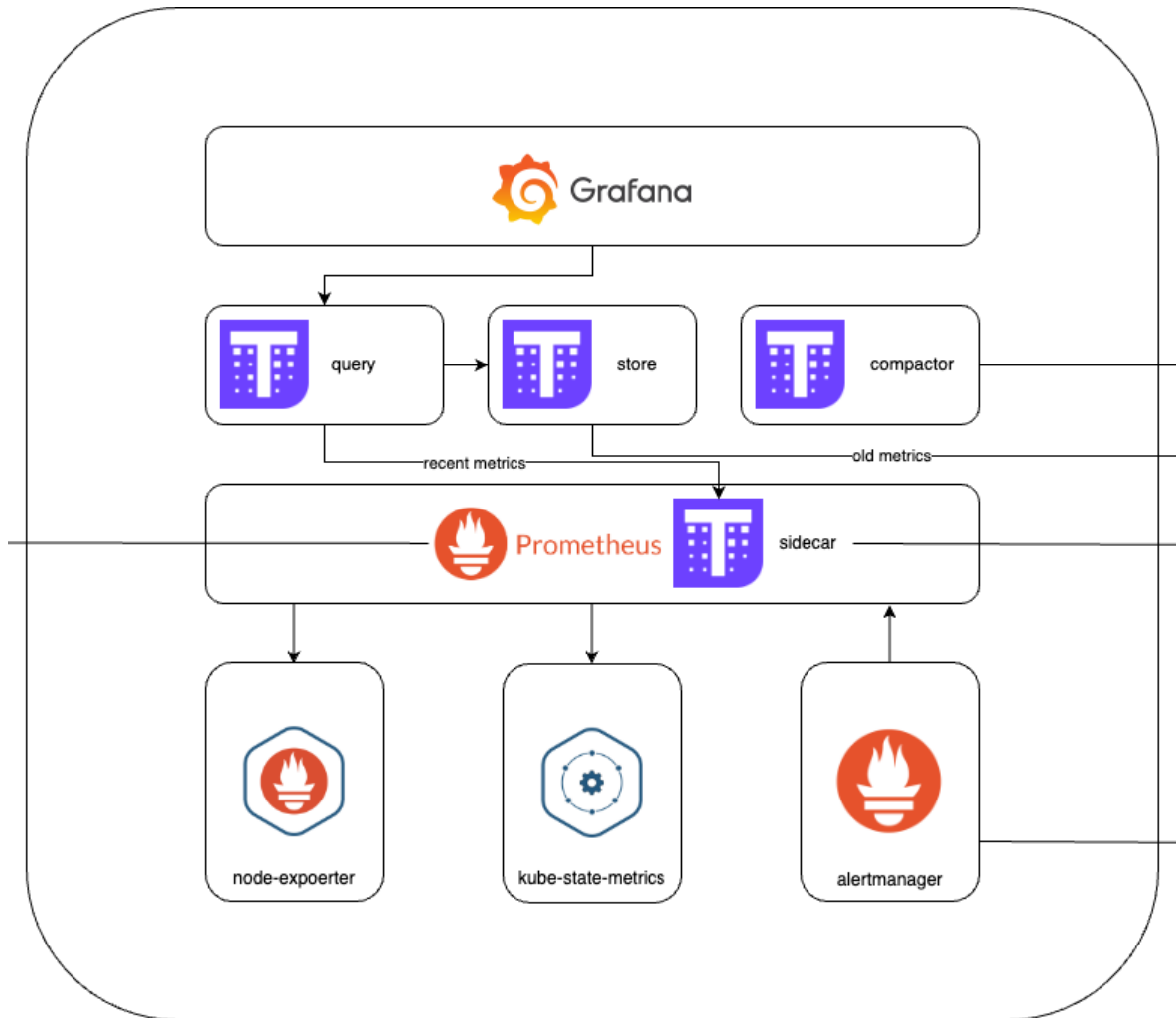


Figure 20: Thanos diagram with Prometheus and Grafana integration, showing that a deployment that incorporates both long and short-term time stores is feasible [29].

- **Predictive Carbon Forecasting:** Integrating historical grid carbon trends and machine-learning forecasting APIs to predict upcoming carbon spikes would allow users (and eventually the custom scheduler) to preemptively queue and execute batch workloads during forecasted off-peak carbon hours.
- **Cost Correlation:** Overlaying standard cloud billing metrics with the existing emission data

would help organisations visually correlate financial cost with environmental impact, quantifying the monetary savings generated by scaling down idle nodes.

- **Additional Grafana Dashboards:** The current Grafana dashboards ConfigMap provides some basic but informative insights into the sustainability metrics of the cluster. However, this could be expanded upon, with potential future dashboards including:
 - **Node-Level Eviction Counters:** Tracking the frequency and geographic location of Day-2 evictions over time, allowing platform engineers to identify which regional grid zones are the most volatile and unsuited for strict **hard** enforcement policies.
 - **Real-time Carbon Footprint and Power Draw by Namespace:** Early prototype dashboards attempting to map these metrics frequently failed (returning “No data”) due to PromQL aggregation latency and the difficulty of attributing unlabelled background processes in Kepler to specific tenant namespaces. Future iterations should implement a refined metric relabelling pipeline to accurately distribute system overhead across tenant boundaries.

8.4 Improved RBAC Roles and Security in Cluster Environment

To rapidly prototype the Day-2 rescheduling capabilities, the current **Rescheduling Controller** utilises a high-privilege **cluster-admin** bypass to issue graceful termination requests via the native Kubernetes Eviction API, which completely violates the principle of least privilege.

Future work requires a security overhaul of the operator’s RBAC manifests. The controller’s permissions must be strictly scoped down to exclusively allow only what the operator needs to perform its reconciliations, including pod evictions. Leaving the operator as is introduces a critical vulnerability since, should the controller be compromised, an attacker could leverage these excessive permissions to bypass namespace isolation and disrupt workloads across the entire infrastructure.

8.5 Other Kubernetes Deployments

At the moment this project is only fully supported in KinD, but all of the manifests in the **demo-yamls** directory can be applied to self-managed vanilla Kubernetes environments, as shown in the A/B test discussed in the previous sections. Future work for this project aims to support the sustainability stack on cloud managed environments, such as Amazon’s EKS and Google’s GKE clusters.

However, transitioning from a self-managed architecture to a managed cloud offering introduces several severe architectural challenges that must be addressed in future iterations:

- **Control Plane Abstraction:** In managed environments, the cloud provider abstracts and entirely manages the Kubernetes control plane (API Server, **etcd**, and the default **kube-scheduler**). Users alter core component flags or directly mount volume paths and because of this, deploying the **susk8s-scheduler** requires running it as a secondary, containerised deployment within the worker node plane, which introduces complexities regarding leader election, RBAC permissions, and potential latency when communicating with the abstracted API server.
- **Hardware Telemetry Limitations (Kepler Constraints):** The SusK8s observability plane relies heavily on Kepler to extract raw power draw metrics via eBPF and hardware performance counters. In virtualised cloud environments, the hypervisor layer can heavily restricts or block access to bare-metal RAPL counters. Future work must investigate calibrating Kepler’s could be installed in these environments and what limitations it brings.

8.6 Directory and File Refactoring

As the project evolved from its initial semester one, several legacy files and scaffolding directories were deprecated (such as the original **internal/controller** package generated by Kubebuilder). In order to not break anything unexpectedly, certain directories have kept their original name and files that are no longer used are still present.

Future maintenance work includes a refactoring of the codebase for clarity and ease of navigation. This involves the renaming of directories such as **prototype** to something more relevant like just **susk8s**

and `demo-yamls` to `manifests` for a more professional and clear look for the project. Regarding the `demo-yamls` directory, the unused manifests have been kept as samples for users to use however, these could technically be cleaned too.

Utilising Kubebuilders deep directory scaffolding would also increase clarity in the codebase, such as properly utilising the `internals` with subdirectories for `controller`, `webhook` and `metrics` as opposed to dumping all of my operator controller, webhook and metrics code in the `controllers` directory.

9 Summary and Discussion

The previously mentioned goals of the SusK8s project was to model sustainable API resources to Kubernetes, reconcile a cluster's state towards sustainable intent and provide carbon-aware scheduling, auditable reports and an observation pipeline that focuses on sustainability, all of which were achieved. The project successfully validates the premise that Kubernetes can be natively extended to enforce environmental constraints without modifying upstream source code. It achieves application-level sustainability users how they want to schedule and reschedule they're sustainable workloads, queries and handles regional grid intensity metrics from external sources, and offers auditable reports, dashboards and visualisation topologies for users to view sustainability metrics within their cluster.

Learning Outcomes The findings of this research contribute a critical insight to the broader field of green cloud computing. The data produced from the A/B test definitively shows that to achieve carbon friendly computing, custom scheduling intelligence must act as the trigger mechanism for physical server scaling. By laying the declarative and observational groundwork, SusK8s provides the exact architectural foundation required for the next set of future work.

Personally, I have also further strengthened my Kubernetes operator knowledge while also learning more about Kubernetes architecture as a whole, especially regarding scheduling within Kubernetes.

Things I Would do Differently: If I were to restart the project, I would implement the Mutating Admission Webhook pattern from Day 1 for all Day-0 policy enforcement. Learning that controllers should be reserved for passive observation/Day-2 reconciliation, while Webhooks must handle strict Day-0 payload interception, was a major architectural lesson that cost significant refactoring time.

I would have also implemented my entire testing suite sooner as well. If I had done this, I would have found out, through my A/B testing scenario, that additional carbon-aware sustainability measures at the hardware level would be required in order to make an actual impact on carbon emissions.

Ultimately, the development of this project was an exercise in shifting from theoretical sustainability to empirical systems engineering. The challenges encountered, such as from the webhook race conditions realisation, highlight the extreme complexity of cloud orchestration. These lessons learned not only shaped the final, robust architecture of the system but also provided the crucial insights needed to define the direction of future carbon-aware infrastructure.

References

- [1] AWS Builders. Your guide to prometheus monitoring on kubernetes with grafana, 2023. Accessed: 2026-05-03. URL: <https://dev.to/aws-builders/your-guide-to-prometheus-monitoring-on-kubernetes-with-grafana-gi8>.
- [2] Kiran Chowdhury. Effortlessly develop and deploy kubernetes custom resources with operator sdk: A step-by-step guide. <https://medium.com/@kiran.soft/effortlessly-develop-and-deploy-kubernetes-custom-resources-with-operator-sdk-a-step-by-step-guide>, 2023. Accessed: 2026-05-01.
- [3] ElectricityMap. Real-time electricity carbon intensity. <https://www.electricitymap.org/>, 2025. Accessed: 2025-12-15.
- [4] Grafana Labs. Grafana: The open observability platform. <https://grafana.com/docs/grafana/latest/>, 2026. Accessed: 2026-03-05.
- [5] Green Agile. Software carbon intensity (sci) score. <https://greenagile.org/metrics-and-impact/code-build-and-cicd/sci>, 2026. Accessed: 2026-05-01.
- [6] Green Software Foundation. Software carbon intensity (sci) specification (iso/iec 21031:2024). <https://greensoftware.foundation/standards/sci/>, 2024. Accessed: 2026-05-05.
- [7] Henning Jacobs. Kubernetes operational view (kube-ops-view). <https://github.com/hjacobs/kube-ops-view>, 2026. Accessed: 2026-03-29.
- [8] Piyush Jajoo. Kubernetes operators: A deep dive into the internals. <https://dev.to/piyushjajoo/kubernetes-operators-a-deep-dive-into-the-internals-221m>, 2026. Accessed: 2026-02-05.
- [9] Karmada Authors. Karmada architecture, 2026. Accessed: 2026-05-01. URL: <https://karmada.io/docs/core-concepts/architecture/>.
- [10] Karmada Authors. Karmada controllers: Orchestrating multi-cluster workloads, 2026. Accessed: 2026-05-01. URL: <https://karmada.io/docs/core-concepts/controllers/>.
- [11] Kubernetes Authors. Custom resource definitions. <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>, 2025. Accessed: 2025-10-15.
- [12] Kubernetes Authors. Disruptions. <https://kubernetes.io/docs/concepts/workloads/pods/disruptions/>, 2025. Accessed: 2025-12-19.
- [13] Kubernetes Authors. Scheduler design overview. <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>, 2025. Accessed: 2025-10-15.
- [14] Kubernetes Authors. Operator pattern. <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>, 2026. Accessed: 2026-05-01.
- [15] Kubernetes SIG Scheduling. Kubernetes enhancement proposal: Scheduling framework. <https://github.com/kubernetes/enhancements/tree/master/keps/sig-scheduling/624-scheduling-framework>, 2025. Accessed: 2025-10-15.
- [16] Alex Leonhardt. Writing a very basic kubernetes mutating admission web-hook, 2019. Accessed: 2026-04-1. URL: <https://dev.to/ineedale/writing-a-very-basic-kubernetes-mutating-admission-webhook-5b1>.
- [17] Weiwei Lin, Tianhao Yu, Chongzhi Gao, Fagui Liu, Tengyue Li, Simon Fong, and Yongxiang Wang. A hardware-aware cpu power measurement based on the power-exponent function model for cloud servers. *Information Sciences*, 547:1045–1065, 2021. Accessed: 2025-12-09. doi:10.1016/j.ins.2020.09.033.

- [18] Dave Masselink. Average or marginal? navigating intensity signals in carbon-aware computing. <https://medium.com/@davemasselink/average-or-marginal-navigating-intensity-signals-in-carbon-aware-computing-3f571bc655ff>, 2025. Accessed: 2026-05-05.
- [19] Microsoft Tech Community. Zero-trust kubernetes: Enforcing security & multi-tenancy with custom admission webhooks. <https://techcommunity.microsoft.com/blog/azureinfrastructureblog/zero-trust-kubernetes-enforcing-security--multi-tenancy-with-custom-admission-we/4466646>, 2026. Accessed: 2026-02-14.
- [20] Platform Engineering. Policy as code: The platform engineer’s guide to automated governance and compliance. <https://platformengineering.org/blog/policy-as-code>, 2026. Accessed: 2026-02-14.
- [21] M. Prashanthi and B. Lalitha. Analyzing dynamic workload management and energy-efficient task scheduling in cloud computing. In *Proceedings of the International Conference on Computational and Embedded Systems (ICCES)*. IEEE, 2024. Accessed: 2025-12-12. doi:10.1109/ICCES63552.2024.10860200.
- [22] Prometheus Authors. Prometheus: Monitoring system and time series database. <https://prometheus.io/docs/introduction/overview/>, 2025. Accessed: 2025-12-15.
- [23] Suzanne Rivoire, Manasi Shah, Juan Meza, Rodrigo Azevedo, Satish Singh, Cenk Isci, and Frank Bellosa. A comparison of high-level full-system power models. <https://rivoire.cs.sonoma.edu/pubs/rivoire.2008.comparison.hotpower.pdf>, 2008. Accessed: 2025-10-14.
- [24] Sustainable Computing IO Project. Kepler: Kubernetes efficient power level exporter. <https://github.com/sustainable-computing-io/kepler>, 2025. Accessed: 2025-10-15.
- [25] Testkube. Policy-as-code in kubernetes: Automating compliance without slowing devops. <https://testkube.io/blog/policy-as-code-kubernetes-compliance-devops-automation>, 2026. Accessed: 2026-02-14.
- [26] Israel Ulelu. Creating a cli in go using cobra, 2024. Accessed: 2026-04-2. URL: https://www.jetbrains.com/guide/go/tutorials/cli-apps-go-cobra/creating_cli/.
- [27] United Nations. The 17 goals — sustainable development, 2015. Accessed: 2026-05-03. URL: <https://sdgs.un.org/goals>.
- [28] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*. ACM, 2015. Accessed: 2025-10-12. doi:10.1145/2741948.2741964.
- [29] Leonardo Wolo. Prometheus high availability with thanos in kubernetes, 2023. Accessed: 2026-05-06. URL: <https://acagroup.be/en/blog/prometheus-high-availability-with-thanos-in-kubernetes/>.